Software Development Lifecycle

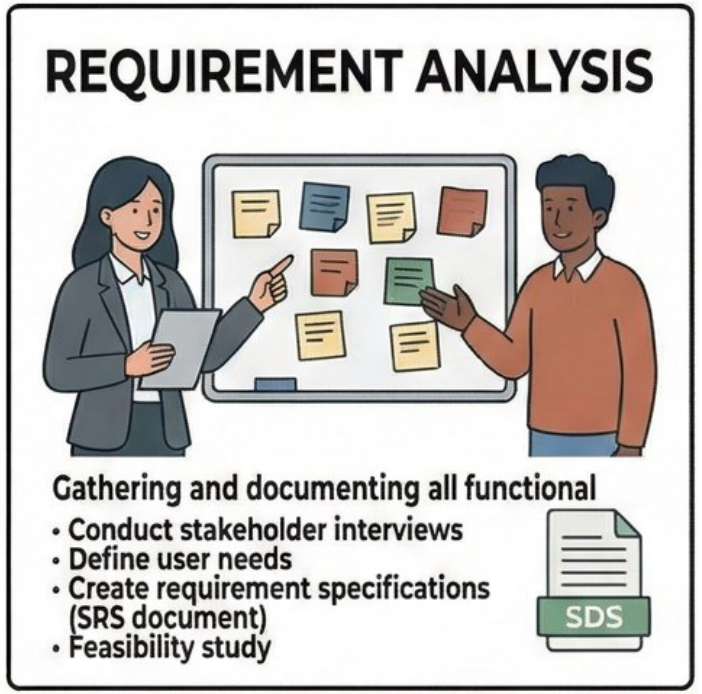
Software Development Life Cycle (SDLC)

- The Software Development Life Cycle (SDLC) is a structured process that outlines the stages of software development.
- Developed in the 1970s to manage large-scale business systems and remains relevant today.
- Stages
  - Requirement analysis
  - System Design
  - Implementation
  - Unit Test
  - Integration Testing
  - System Testing
  - Acceptance Testing
  - Deployment
  - Maintenance

Video: Software Development Life Cycle: Explained

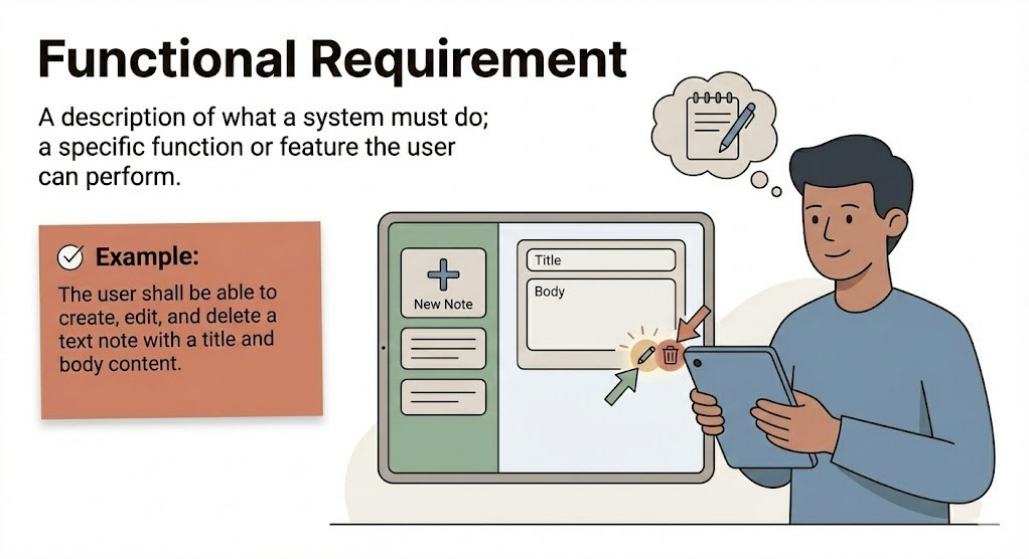
Types of software requirements

- Functional requirements Define what a system is supposed to accomplish
- Non-functional requirements Define how well a system will operate



References:
https://www.altexsoft.com/blog/functional-and-non-functional-requirements-specification-and-types
Video: https://www.youtube.com/watch?v=2qlcY9LkFik

Examples of functional requirements



- The system sends a confirmation email when a new user account is created.
- The system sends an approval request after the user enters personal information.
- A search feature allows users to search content/items by entering the query in the search bar.
- The user can review items in the cart, change their number, or remove them before checkout.
- The app should allow users to create accounts and log in using credentials like email and password or through social media integration.
- The app can send notifications to users for updates, reminders, or promotional content.
- Users should be able to provide feedback or rate services/products within the app.

Collecting functional requirements

Document Analysis

Reviewing existing documentation, such as business processes and systems, to identify requirements.



Observation

Watching users in their environment to understand their workflows and pain points.



Interviews

Conducting one-on-one discussions with stakeholders to gather detailed information.



Surveys and Questionnaires

Distributing structured forms to a large audience to gather quantitative data.



Focus Groups

Engaging a diverse group of stakeholders in discussions to gather a range of perspectives.



Prototyping

Creating early versions of the software to visualize requirements and gather feedback.



Use AI to generate software requirements

SAGE Hong Kong, also known as The Hong Kong Society for the Aged, is an organization dedicated to providing integrated home care services for the elderly in Hong Kong. It focuses on supporting the well-being and quality of life for senior citizens through various community care initiatives. Their services include personalized care plans assisting older adults to live independently at home, emphasizing both physical and emotional support. SAGE plays an important role in elderly welfare within the region, helping to meet the complex needs of the aging population with a compassionate approach.

Generate the functional requirements for a system for SAGE

9:37 PM

GPT-4.1-mini

Here are the **functional requirements** for a system designed for SAGE Hong Kong (The Hong Kong Society for the Aged), focusing on supporting their integrated home care services for the elderly:

Functional Requirements for SAGE Hong Kong Elderly Care System

1. User Management

- FR1.1: The system shall allow registration and management of different user roles such as elderly clients, caregivers, social workers, and administrators.
- FR1.2: The system shall provide secure login and authentication for all users.
- FR1.3: The system shall allow profile creation and editing for elderly clients, including personal details, medical history, and care preferences.

2. Care Plan Management

- FR2.1: The system shall enable social workers/caregivers to create, update, and manage personalized care plans for each elderly client.
- FR2.2: The system shall allow inclusion of multiple aspects in care plans, such as physical care tasks, medication schedules, emotional support activities, and appointments.
- FR2.3: The system shall notify caregivers and elderly clients about upcoming tasks or appointments.

3. Scheduling and Task Management

- FR3.1: The system shall allow caregivers to schedule home visits and care activities.
- FR3.2: The system shall provide task lists for caregivers with details about daily care activities.
- FR3.3: The system shall allow caregivers to mark tasks as completed and add notes or observations.
- FR3.4: The system shall support reminders and alerts for both caregivers and elderly clients.

4. Communication and Support

- FR4.1: The system shall provide a secure messaging platform for communication between elderly clients, caregivers, and social workers.
- FR4.2: The system shall support video calls or teleconferencing for remote emotional support or consultations.
- FR4.3: The system shall enable social workers to broadcast community care announcements or health tips to elderly clients.

5. Health Monitoring

- FR5.1: The system shall allow input and tracking of basic health metrics (e.g., blood pressure, glucose levels) recorded by caregivers or elderly clients.
- FR5.2: The system shall generate alerts if health metrics go beyond predefined thresholds to notify caregivers or medical personnel.

6. Resource and Service Directory

- FR6.1: The system shall provide a directory of relevant community resources, health services, and emergency contacts.
- FR6.2: The system shall allow social workers to update and manage the resource directory.

7. Reporting and Analytics

- FR7.1: The system shall generate reports on care activities, client health status, and service usage for administrators.
- FR7.2: The system shall provide analytics on the effectiveness of care plans and identify clients needing additional support.

8. Emergency Handling

- FR8.1: The system shall have an emergency alert function that elderly clients or caregivers can trigger to notify emergency contacts and social workers.
- FR8.2: The system shall log emergency events and responses for review.

9. Data Privacy and Security

- FR9.1: The system shall ensure all personal and health data is encrypted and accessible only to authorized users.
- FR9.2: The system shall provide audit trails for data access and modifications.

10. Accessibility

- FR10.1: The system shall have a user-friendly interface tailored for elderly users, including options for larger fonts, simple navigation, and voice commands.
- FR10.2: The system shall support multiple languages, including Cantonese and English.

If you want, I can also provide **non-functional requirements** or a more detailed breakdown for specific modules!

Do you think the proposed requirements are what the client needs?

Requirements elicitation

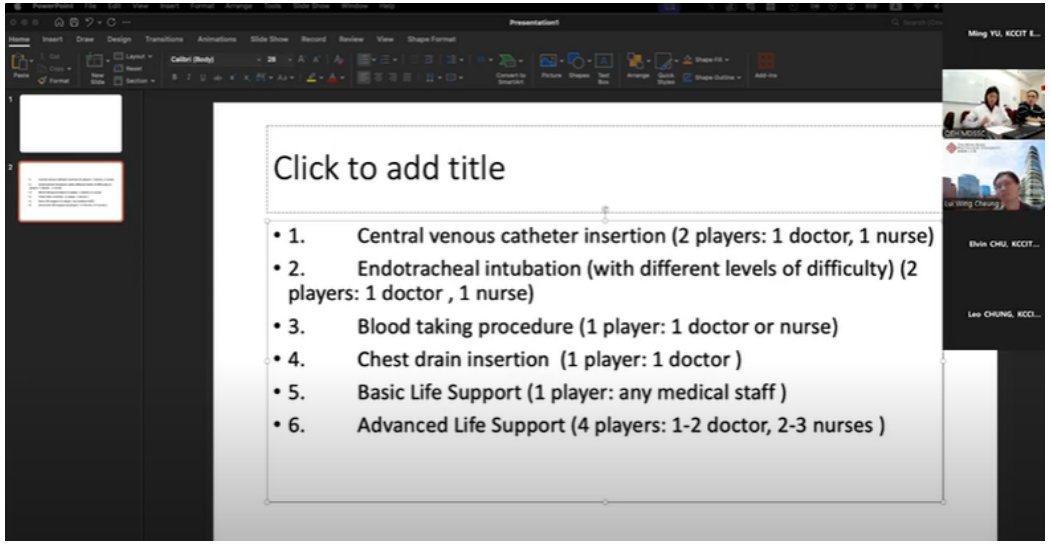
- Define stakeholders' specific needs and expectations for the system
- Understand what needs to be done.
- Tasks:
  - **Identify stakeholders**: determine who will be affected by the project and who should provide input.
  - **Collect information**: conduct interviews, surveys, workshops, etc to gather information about the needs and expectations of stakeholders.
  - **Document requirements**: record the collected information in the documentation formats described above.

Requirements management

- Ensure that all requirements are met during the development process and that any changes are carefully controlled and documented.
- The key components of requirements management are
  - **Prioritization** – determine which requirements are most essential and should be implemented first;
  - **Change management** – handle changes to requirements in a controlled manner to prevent scope creep;
  - **Verification and validation** – ensure that the developed system meets all the identified requirements and that these requirements fulfill their intended purpose.

Examples: Capturing user requirements

- Summarize the requirements from zoom meetings
- Check the AI output for correctness and refine it
- Confirm the refined requirements with the client



GPT-5 ···

Below is a faithful, detailed English translation of the meeting content, organized into sections and focused on the requirements for the six project topics shown in the image. I've preserved the original meaning and nuance, including uncertainties and options discussed.

Section 1. Central Venous Catheter (CVC) Insertion

- Players and roles:
  - Preferably 2 players: 1 doctor (primary operator) and 1 nurse (assistant). The simulation should allow either:
    - Two users collaboratively in the same environment with distinct roles, or
    - A single user (doctor) with the nurse as an AI-controlled character that reacts appropriately to requests and the procedure.
- Rationale and clinical context:
  - Used for unstable surgical or critically ill patients needing vasopressors to raise blood pressure, difficult IV access, or parenteral nutrition when oral intake is not possible.
- Key procedural steps and equipment:
  - Ultrasound-guided vascular access to locate the target vein.
  - Needle insertion into the vein, guidewire advancement, then catheter insertion over the guidewire.
  - Critical safety requirement: removal of the metal guidewire after catheter placement.
- Safety checks and error prevention:
  - A recurring real-world incident is forgetting to remove the guidewire, leaving it inside the patient, which can cause serious complications requiring major surgery.
  - The simulation must emphasize:

Challenges and solutions

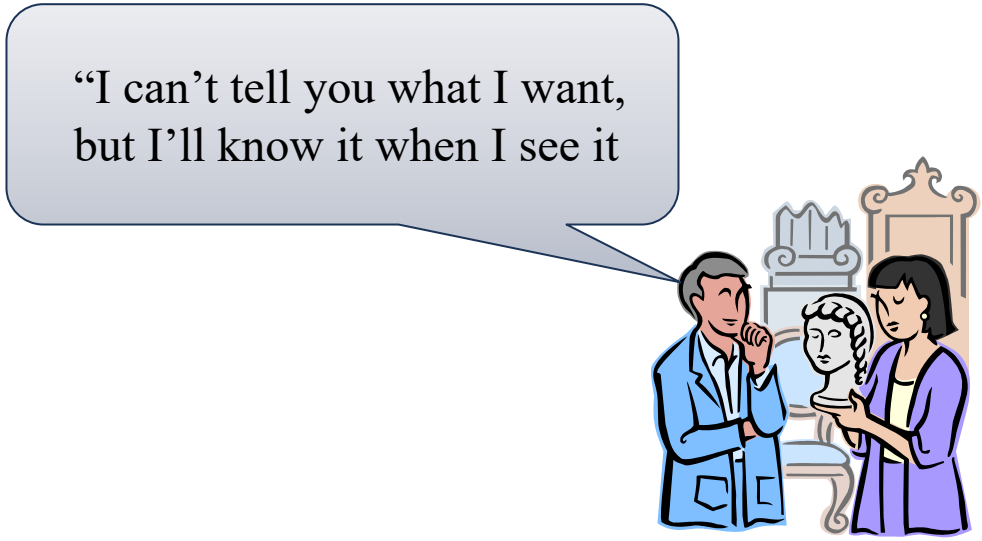
- Challenges
  - Stakeholders struggle to communicate needs clearly
  - Conflicting priorities and assumptions by different stakeholders
  - Business needs shift over time; stakeholders change their minds
- Asking better questions
  - Start with the **problem, not the solution**: “What pain point are we solving? What is the business value?”
  - Ask **open-ended, process-focused questions**: “How do you currently do this? What breaks? What information do you need to decide?”
  - **Challenge assumptions**: “What if data is missing? What are the exceptions? Who owns this decision?”
  - Use **5 Whys** to reach root causes instead of symptoms
- Synthesize, clarify, and resolve conflicts
  - **Paraphrase and confirm understanding** during sessions: “So what I hear is… Is that right?
  - Convert vague statements into **testable, specific requirements** or business rules
  - Use the **MoSCoW** technique (Must/Should/Could/Won’t) to help stakeholders agree on what’s essential now vs later.

Software Prototypes

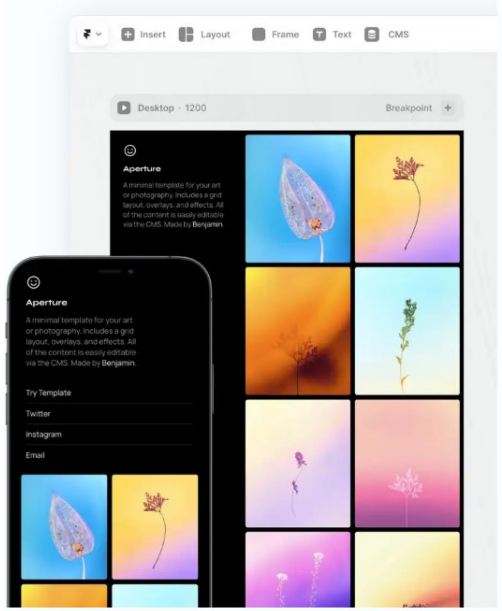
- Software prototype is an umbrella term for different early-stage deliverables that are built to showcase how requirements can be implemented
- **Wireframes** are low-fidelity graphic structures of a website or an app. They help map different product pages with sections and interactive elements.



https://www.altexsoft.com/blog/prototyping



- Mockups
  - Static, but more realistic depictions of the final product
  - Shows the branding, colors, fonts, iconography, and other elements
  - Mockup tools: InVision, Sketch, Figma, Adobe XD, Framer.
- High-fidelity prototype
  - The closest resemblance to the final product, a hi-fi prototype allows for testing both how people react to the product's visuals and user flow

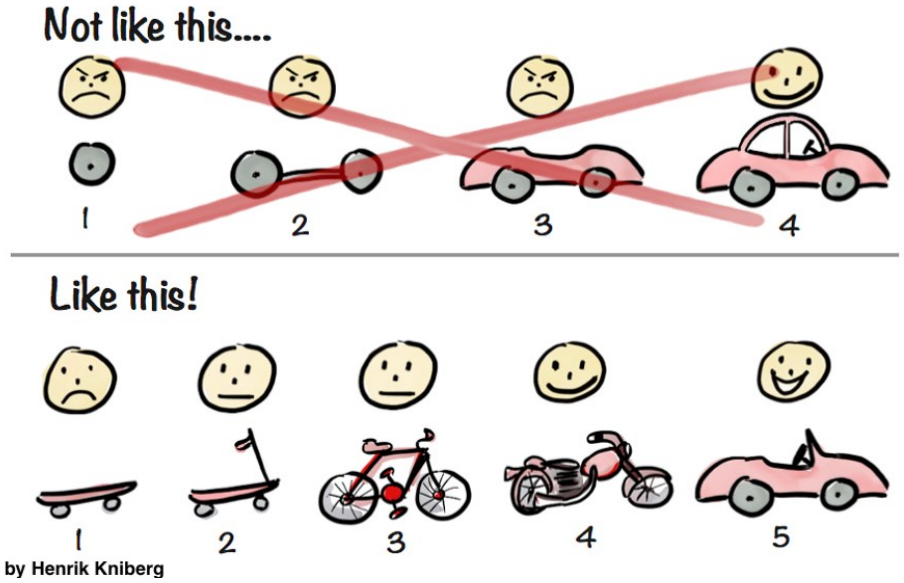


Types of prototype

- Throwaway Prototype
  - Quickly built prototype to gather feedback and refine requirements; discarded after use
- Evolutionary Prototype
  - Incrementally refined based on feedback until final version
- Incremental Prototype
  - Breaks final product into pieces, develops parts separately and integrates at the end

Minimum Viable Product (MVP)

- Market validation is crucial when building something unique and uncertain
- Create a version of a new product which allows a team to collect the maximum amount of validated learning about customers with the least effort
- **Minimum:** spend as little time and effort to create it
- **Viable:** sufficient quality to provide an accurate assessment of whether customers will use the product.

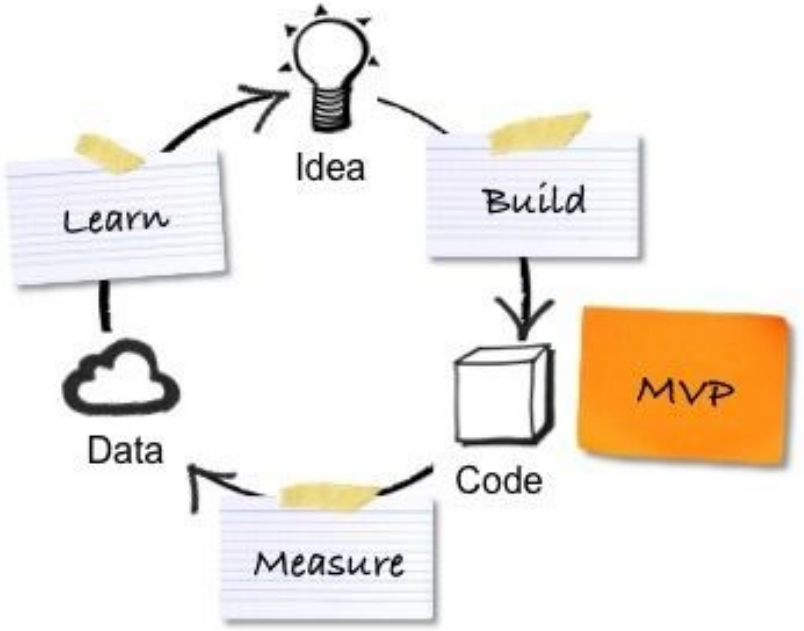


Video Link

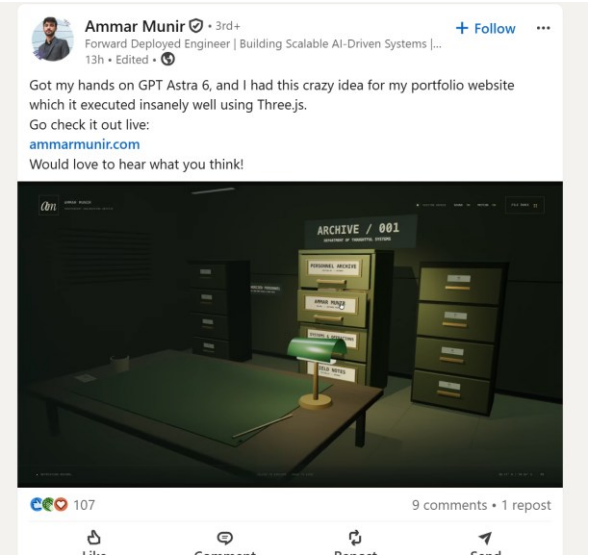
Learning with MVP

- Test an idea by exposing an early version of your product to the target users and customers, to collect the relevant data, and to learn form it.
- Reduce risk of developing a product that is not economically viable.
- Once the MVP is on the market, you collect your key value metrics, create the next MVP, and so on.

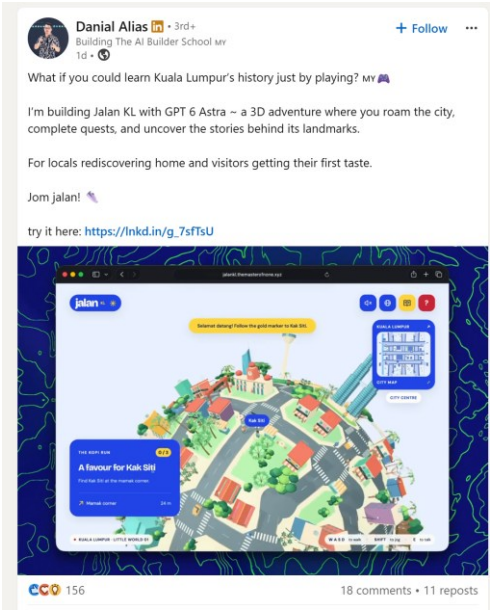
Aspect	Prototype	MVP (Minimum Viable Product)
Purpose	Test and visualize concept, design, and UX	Deliver core functionality to users
Functionality	Often non-functional or limited functionality	Fully functional with core features
Market readiness	Not market-ready	Market-ready
User feedback focus	Initial feedback on design and feasibility	Real user feedback on product value and usability
Development stage	Early stage, before full development	Early launch phase, first usable version
Goal	Validate design ideas and technical feasibility	Validate business idea and attract users



Prototyping with AI



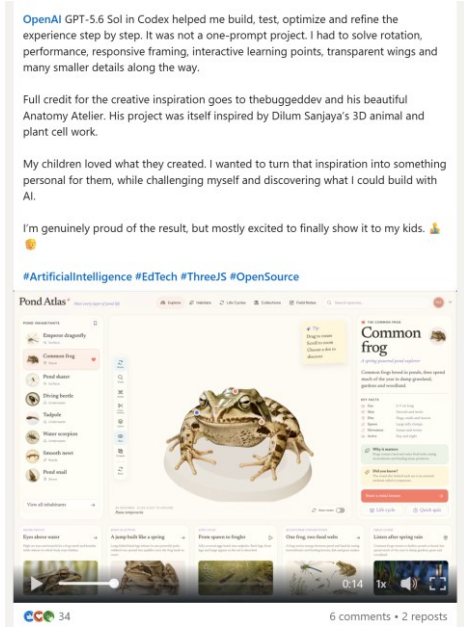
https://lnkd.in/p/edNRmAsy



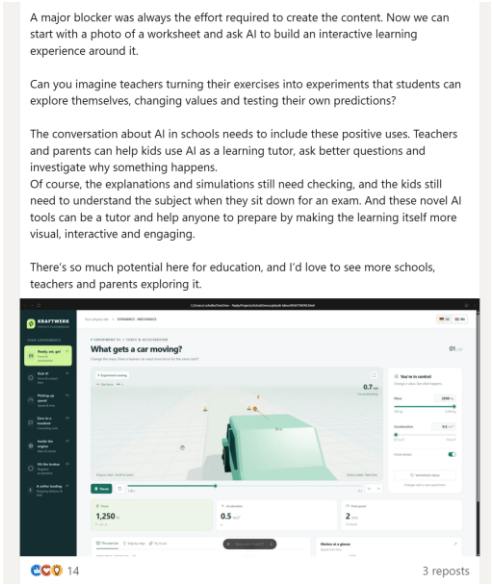
https://lnkd.in/p/eWFBdhWH



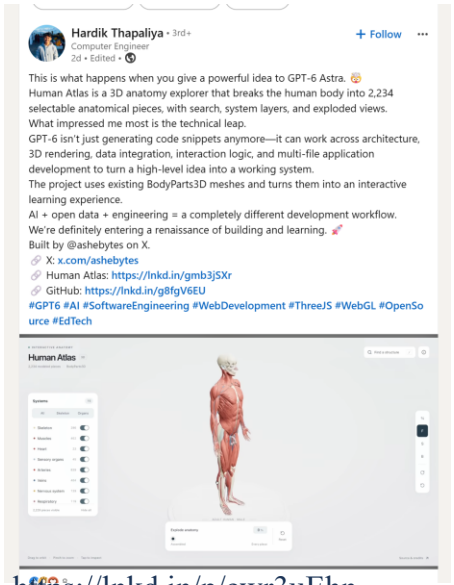
https://www.linkedin.com/posts/emollick_i-had-early-access-and-a-longer-post-is-activity-7501366954893529089-NQ2E



https://lnkd.in/p/eAYFM_DQ



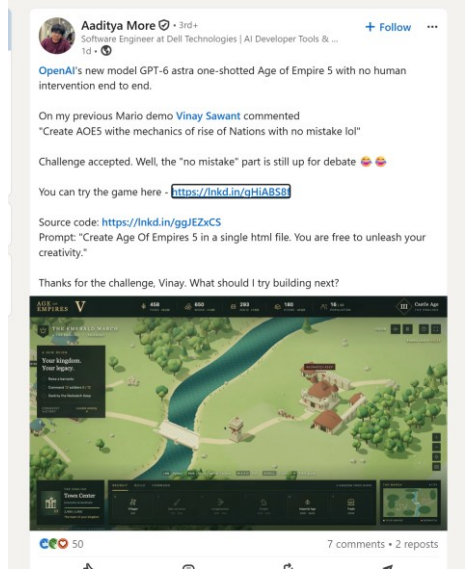
https://lnkd.in/p/eZdR6ySW



https://lnkd.in/p/ewr3uEhn

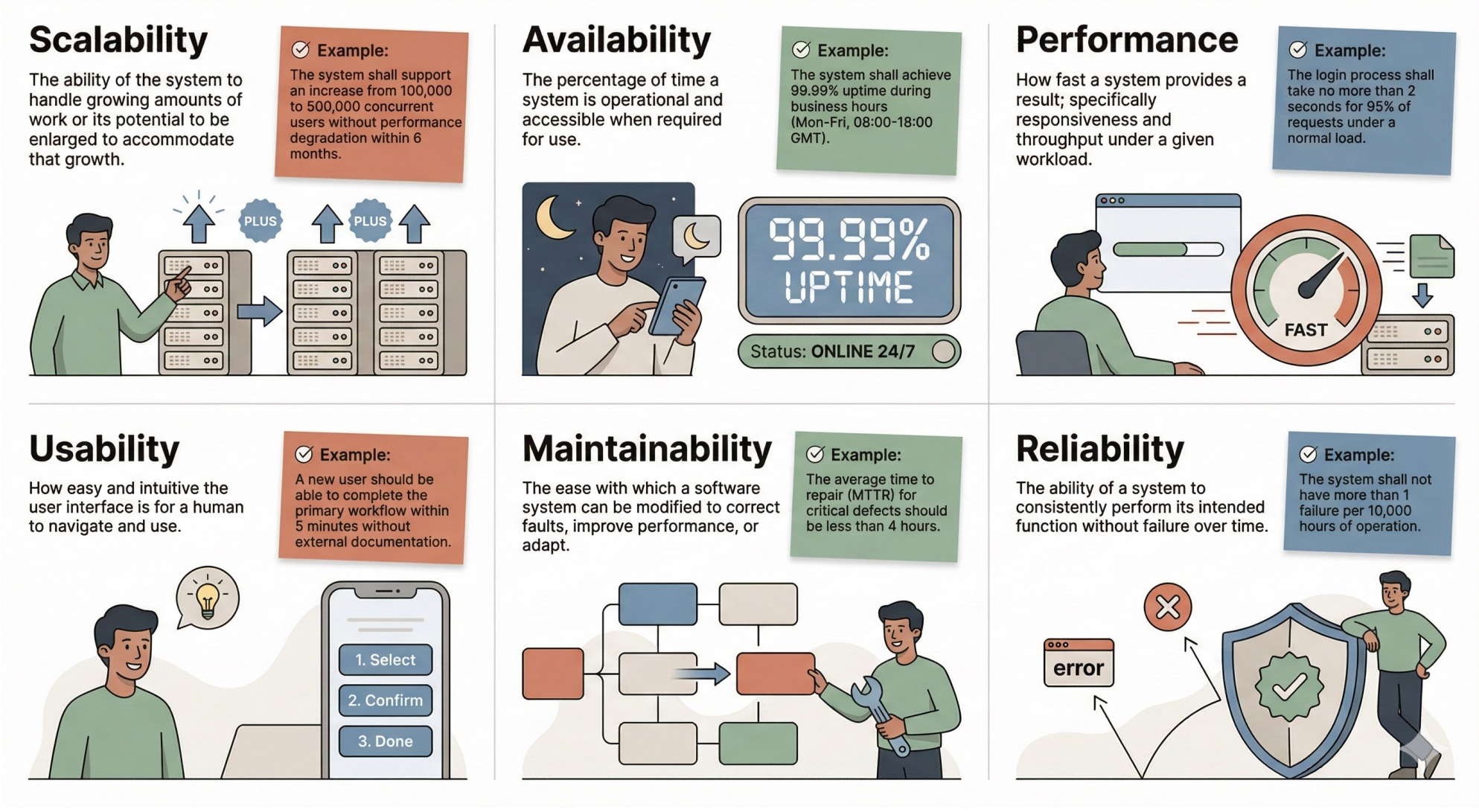


https://lnkd.in/p/ebPqJ-Ez



https://lnkd.in/p/eghGWaJw

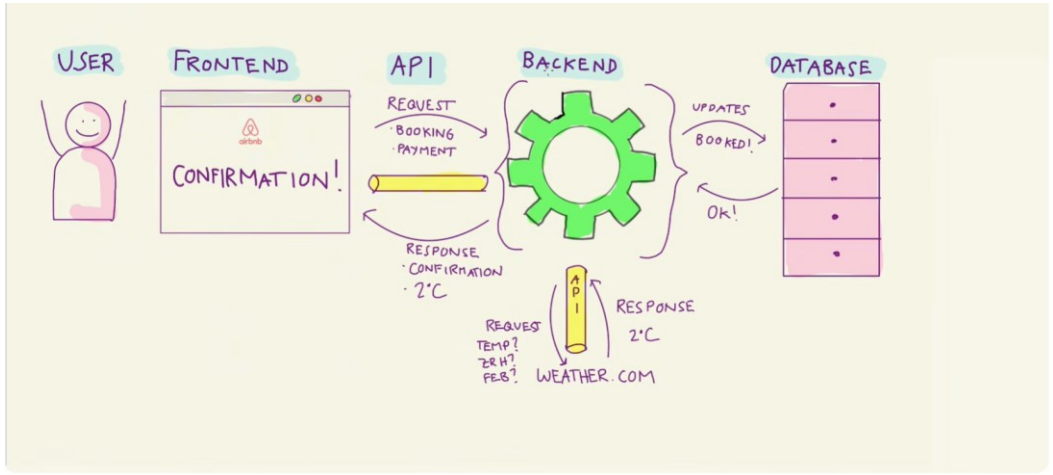
Non functional requirements



https://www.altexsoft.com/blog/non-functional-requirements
Video: What are Non-functional Requirements and How Do They Work?

Design and Implementation

- The requirement specifications from the first phase are studied in this phase, and the software system design is prepared.
- Requirements should state **what** the system should do and the design should describe **how** it does this.
- **System design interview:** a technical assessment in which a candidate is asked to architect a scalable software system on a whiteboard or virtual canvas to demonstrate their ability to reason about requirements, trade-offs, and component interactions.

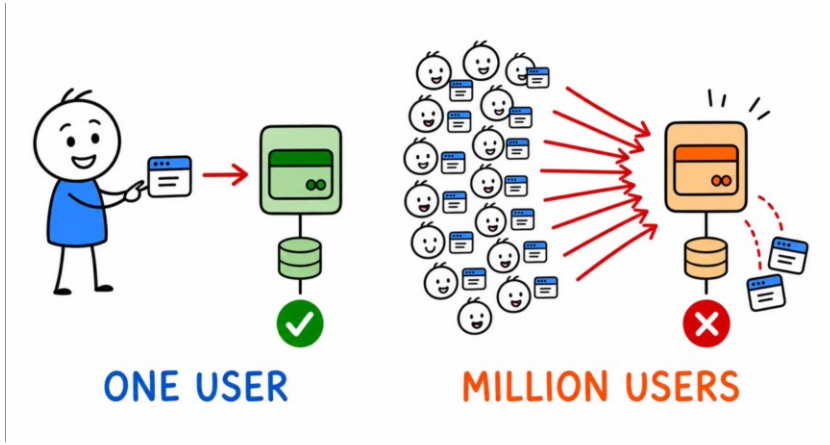


Frontend, API, Backend and Database explained

Tamara Jost
2.33K subscribers
Subscribe
3.3K
NotebookLM
Share
Copy transcript

119K views 1 year ago
A simple breakdown of how the frontend, backend, database, and API work together to create a seamless user experience – an essential part of the technical skill set every product manager needs.

https://www.youtube.com/watch?v=NzEYYemQ3_8



System Design Fundamentals You Should Know | Under 15 Minutes

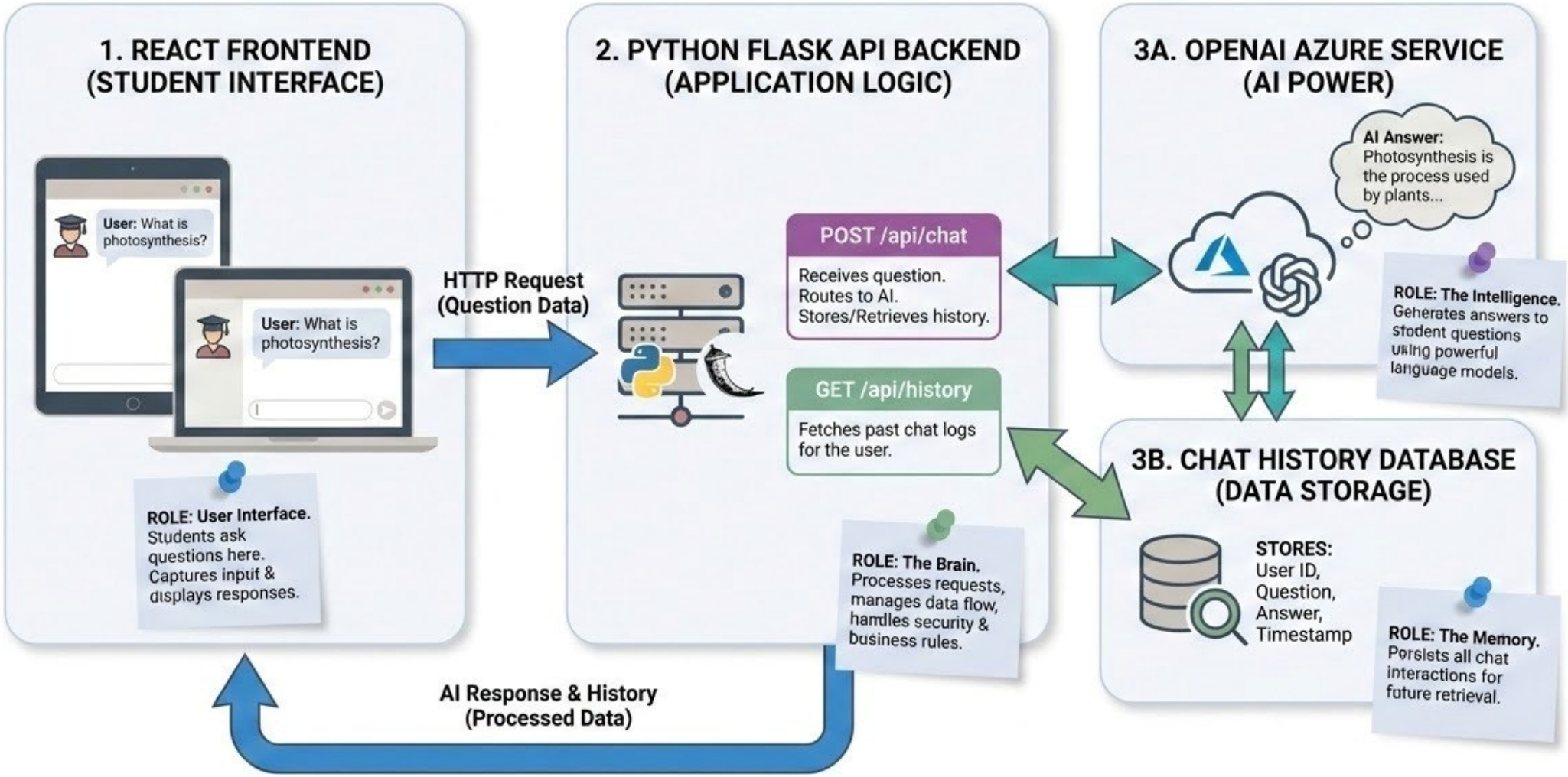
TechGik
810 subscribers
6
NotebookLM
Share
Save
Copy transcript

196 views Aug 14, 2026 #SystemDesign #SoftwareEngineering #Scalability
A system that works for one user may fail completely when a million users arrive.

In this video, we explore the system design fundamentals behind scaling a simple application from zero to millions of users. We begin with a single server and evolve the architecture step by step. At each stage, we identify the next bottleneck, understand why it appears, and introduce the component that addresses it.

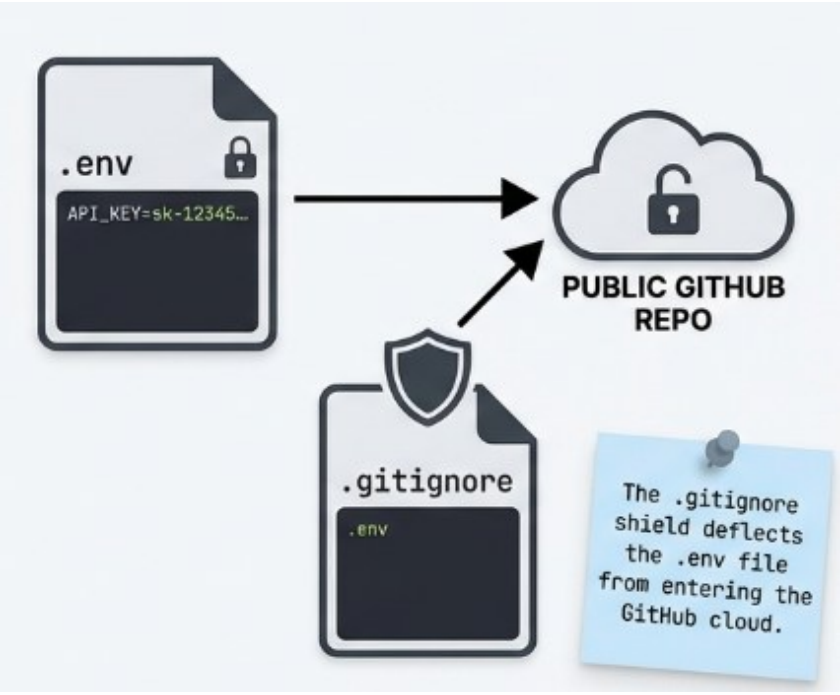
https://www.youtube.com/watch?v=QiMRDD8OUqs

Example: GenAI Powered Chat App



- **Python Flask API Backend**
  - An API (Application Programming Interface) is a set of rules and tools that allows different software applications to communicate and interact with each other.
  - The backend handles business logic, stores, and retrieves data from the database
    - E.g. Receives questions from the frontend via API calls, sends them to the AI service, and returns answers.
- **React Frontend**
  - The user interface for the students to type questions and view answers.
  - It sends questions to the backend using API requests and displays the responses.
- **OpenAI Azure Service**
  - The GenAI API answers using advanced language models.
- **Database**
  - Stores past chat interactions

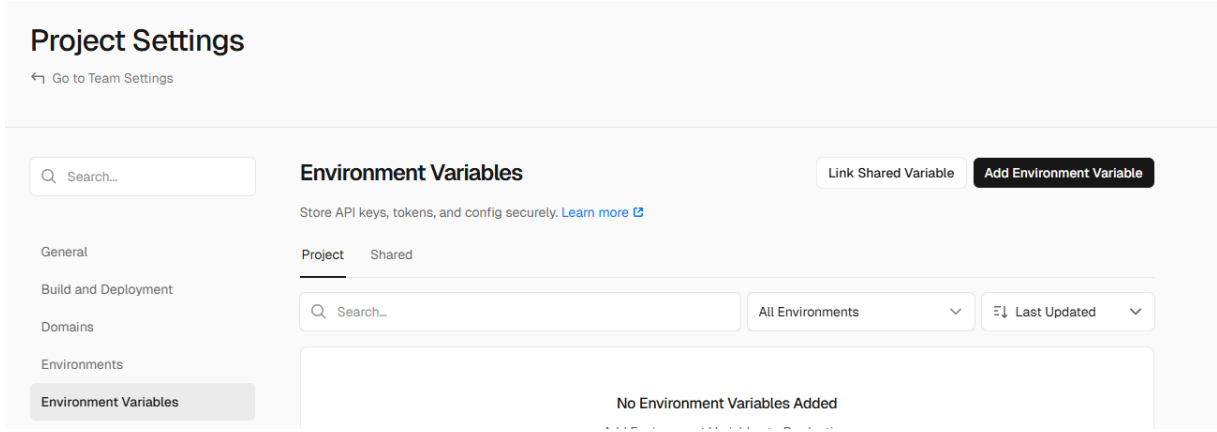
- Protecting the API keys
  - The **.env** file stores secret keys as local environment variables for development
  - The **.gitignore** file ensures that .env and other sensitive files are excluded from version control, preventing them from being pushed to public repositories like GitHub.
  - Secret keys and sensitive data must NEVER appear in the remote repository

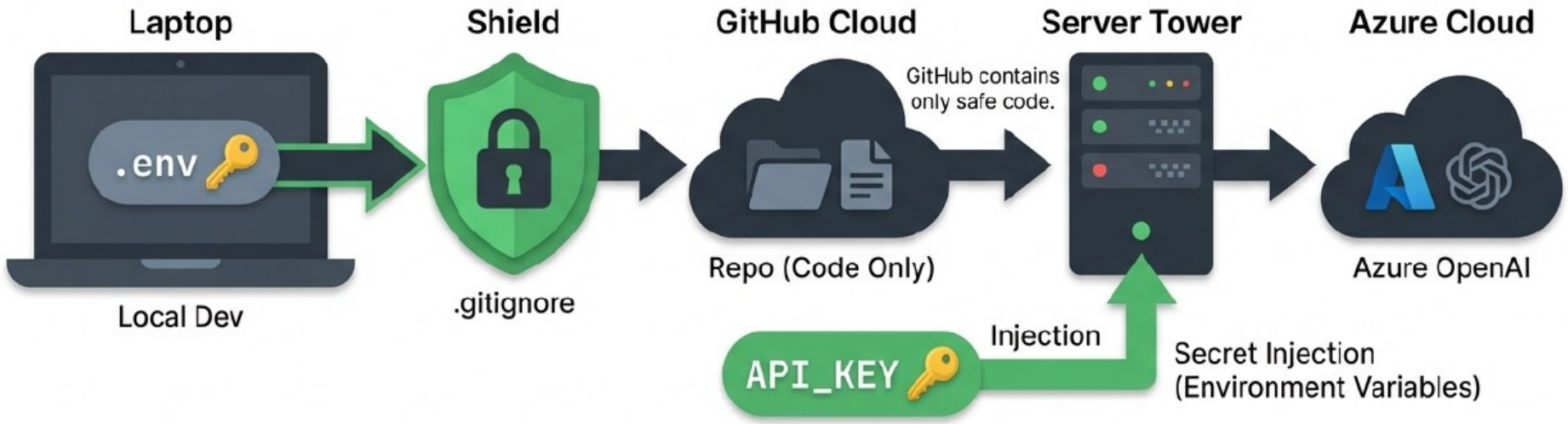


GitHub

src/	Remove mul and div	2 years ago
app.py	Init Repository	2 years ago
package.json	Init Repository	2 years ago
.gitignore	Init Repository	2 years ago
README.md	Init Repository	2 years ago
.env	JetBrains Mono	NOT present in remote history.

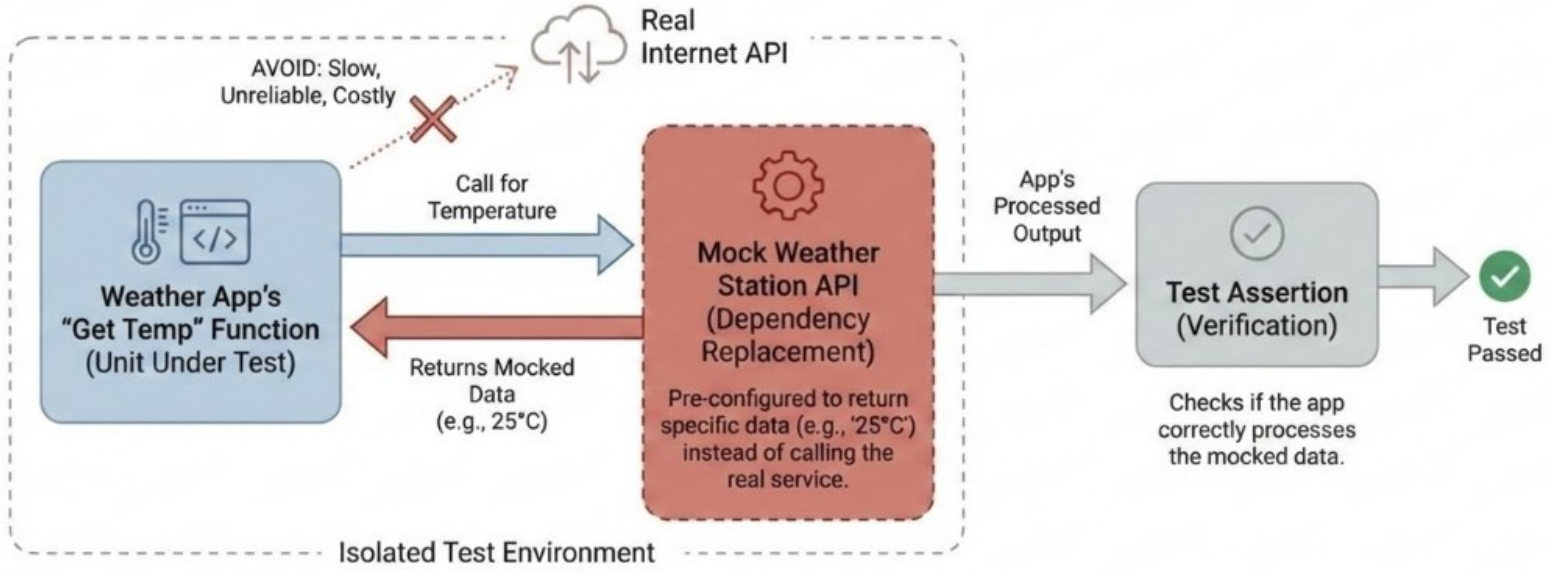
- API key in production
  - In production, API keys are securely stored in online platforms
    - e.g. Vercel's environment variable settings
  - Secret keys are injected into the running application during deployment
  - API Keys remain hidden from the public and frontend code.





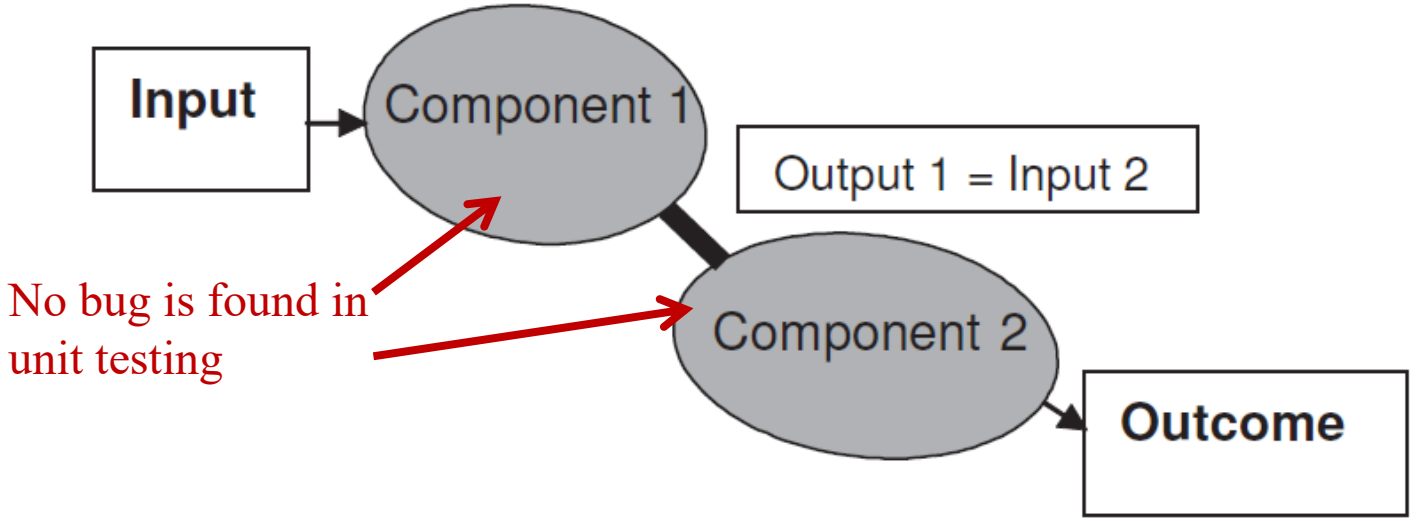
Implementation and Unit Testing

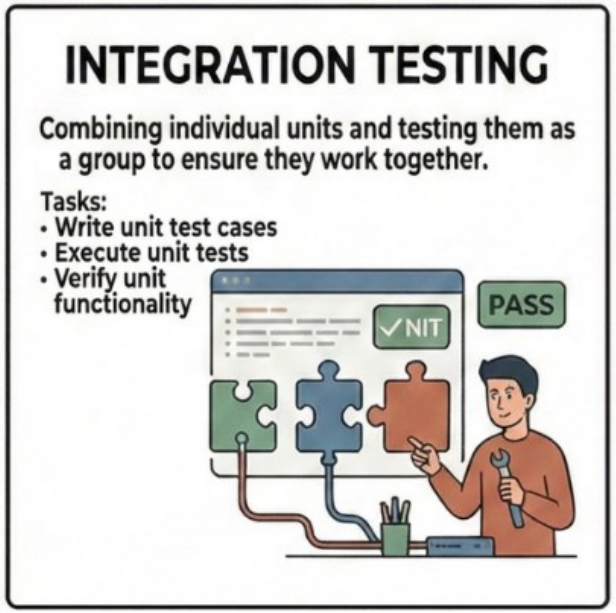
- Individual units are tested in isolation and separated from other parts of the system
  - e.g., procedures, functions, methods, or classes
- Testers verify that an individual component or unit implements all design decisions made in the unit's design specification.
- Units are tested in isolation. Easy to identify faults in individual units.



Integration testing

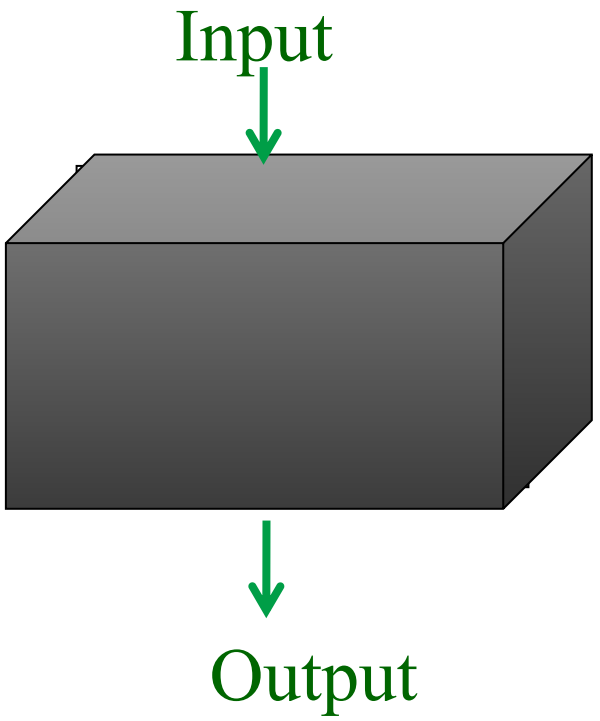
- Individual software modules are combined and tested as a group
- Even though a module is correct and bug-free, bug may arise when we integrate two or more modules together.
  - E.g. A calling module may misunderstand the interface specification of a called module

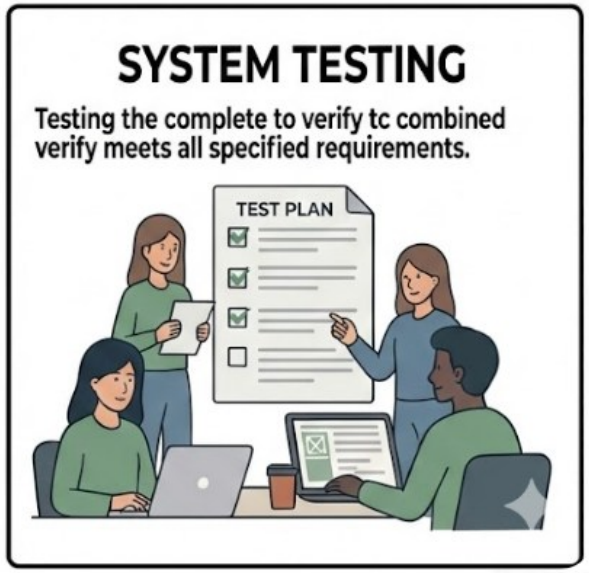




System testing

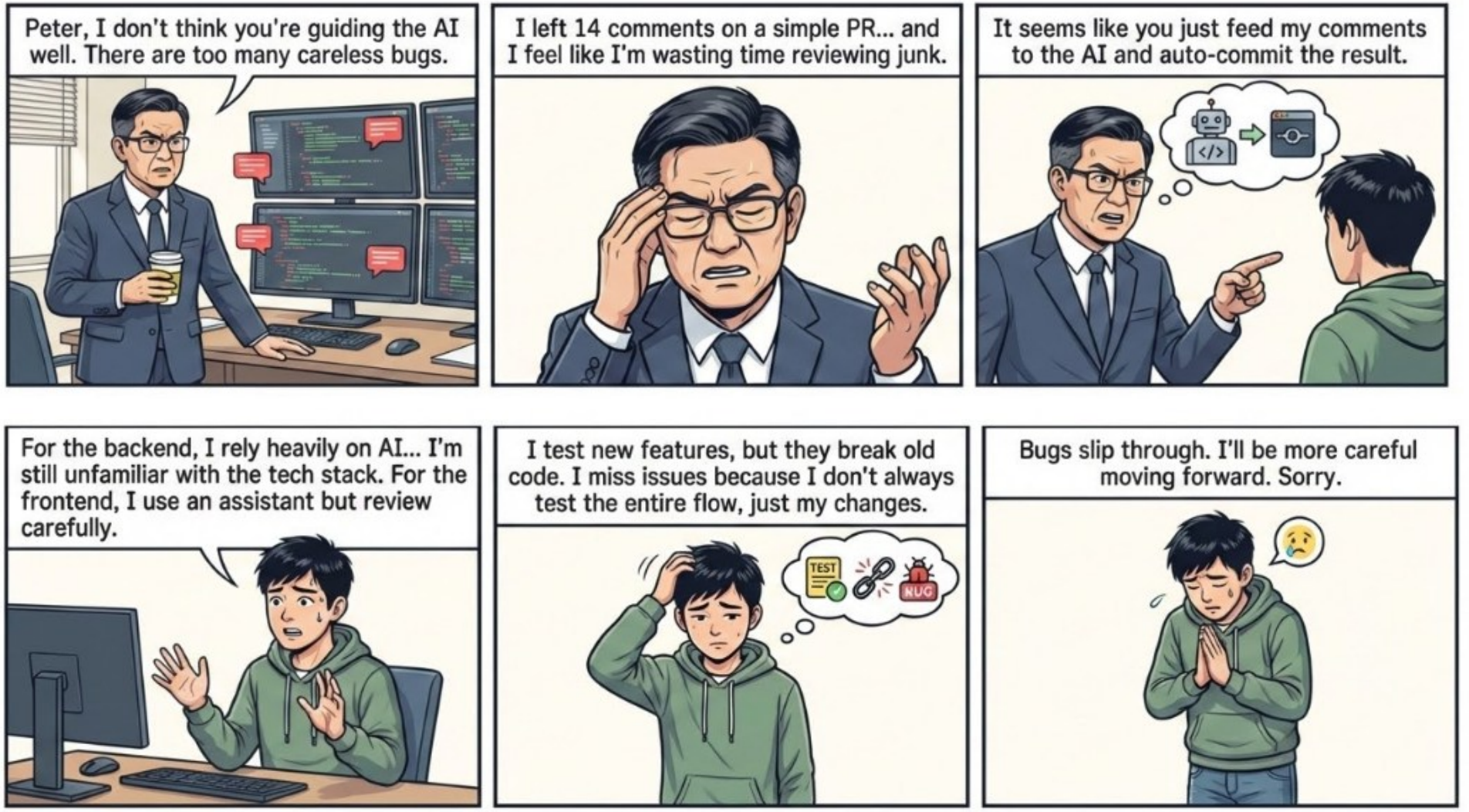
- Verifies that the system implements all the requirements specified in the requirement specification
- Examples
  - Functionality test: Implement all the required features
  - Robustness test: Ability to recover from error conditions
  - Interoperability test: Interoperate with other third-party systems
  - Performance test: Throughput, response time, etc
  - Scalability test: How the system behaves with the increasing scale (e.g. number of users)
  - Reliability: The system should keep operating over a long period of time reliably
  - Documentation test: Ensure that the system's user guide is accurate and usable



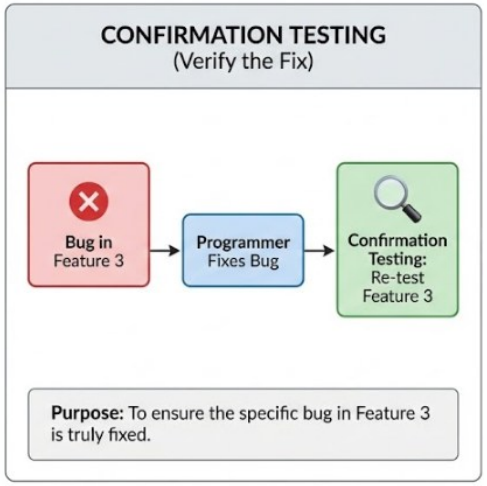


The Pull Request (PR) Problem

A **pull request** is a software development workflow in which a developer proposes merging changes from one branch into another, enabling teammates to discuss, review, and approve the modifications before integration.

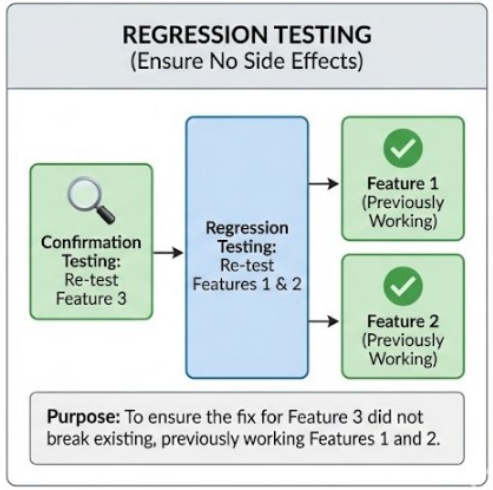


Confirmation vs. Regression Testing



Confirmation testing

- Testing that runs test cases that failed the last time, in order to verify the success of the correctness action.

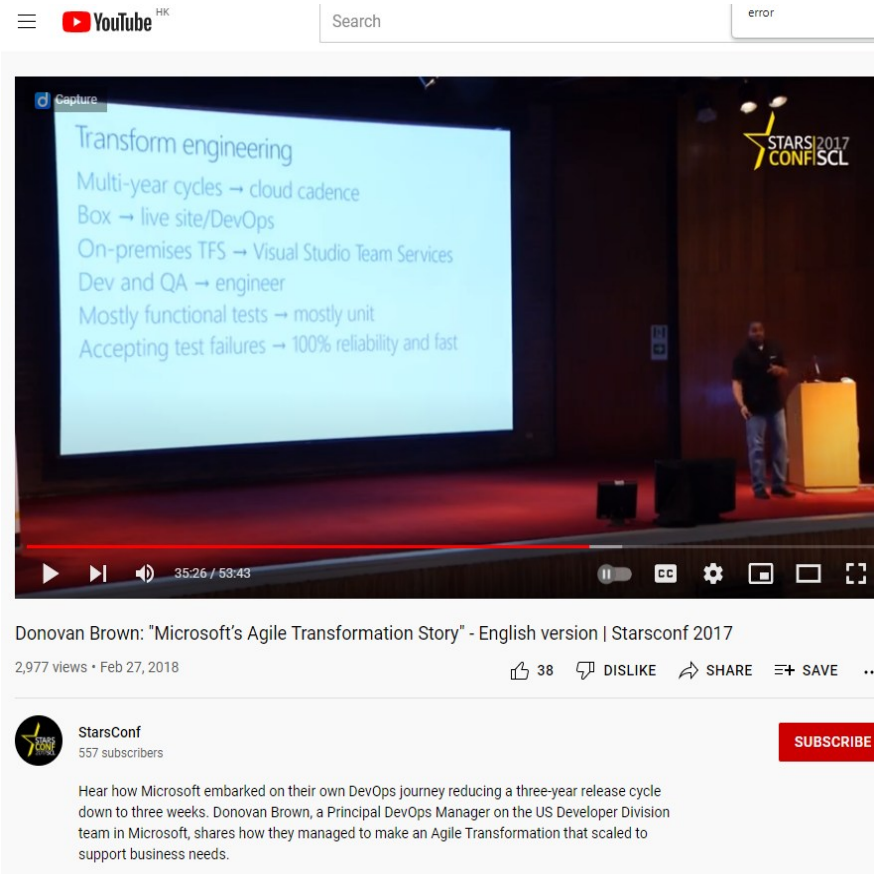


Regression Testing

- **Regression**: The misbehavior of a previously correct function, attribute, or feature
- Testing of a previously tested program following modification (e.g., after bug fixing or adding new features)
- Existing test cases are re-run to ensure that defects have not been introduced or uncovered in the unchanged areas of the software as a result of the change.
  - Re-run all existing test cases: Test automation may speed up regression testing.
  - Select a subset of features for re-testing
    - E.g. Run only tests that reach or are related to the modified statements/modules

Test Automation

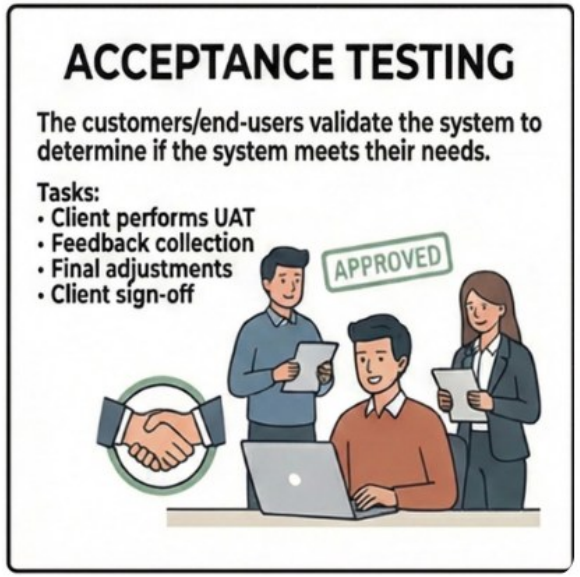
- Test automation involves the use of software to control the execution of tests
- A unit test case consists of code which checks if another part of code (method) works as expected.
- JUnit is a unit testing framework for Java programming language.
- Automated Testing at Microsoft
  - “We literally run 60,000 plus **unit tests** on every single pull request”
  - “We run sixty somewhat thousand unit tests in under seven minutes”



Donovan Brown: "Microsoft’s Agile Transformation Story"
https://youtu.be/nno6DwevspI

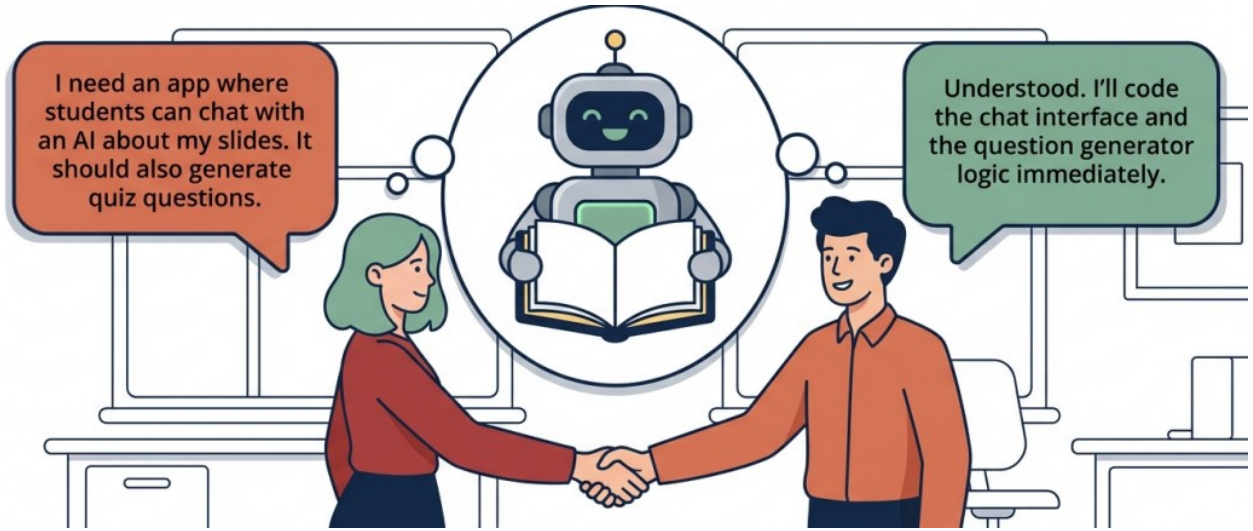
Acceptance Testing

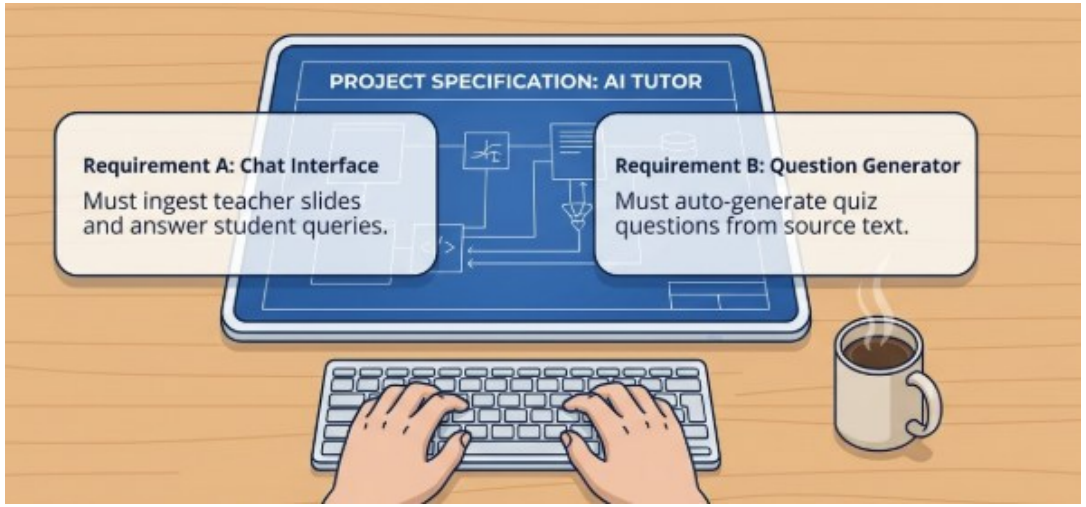
- The customer/end-users validate the system to determine if the developer has created the right system based on the acceptance criteria.
  - Functionality, Performance, Security, etc
- Type
  - Alpha Test
    - Conducted at the developer's site by the users with the developer's presence
  - Beta Test
    - Conducted at a number of user sites without a developer.
    - The testing is more likely to mimic the actual use of the system.



Example: Developing a learning app

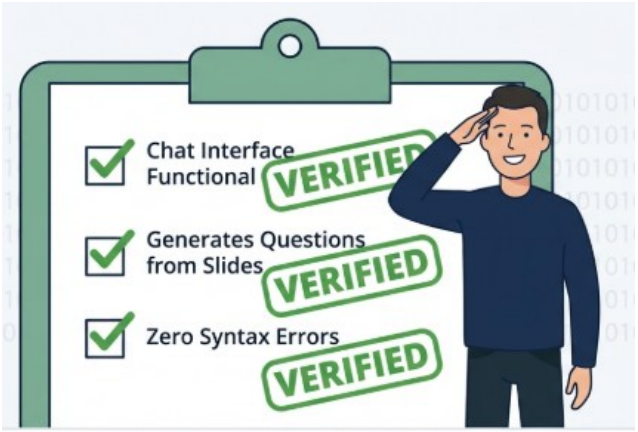
- "The project begins with a set of requirements.
- The goal is clear: create an AI tutor to support student learning."

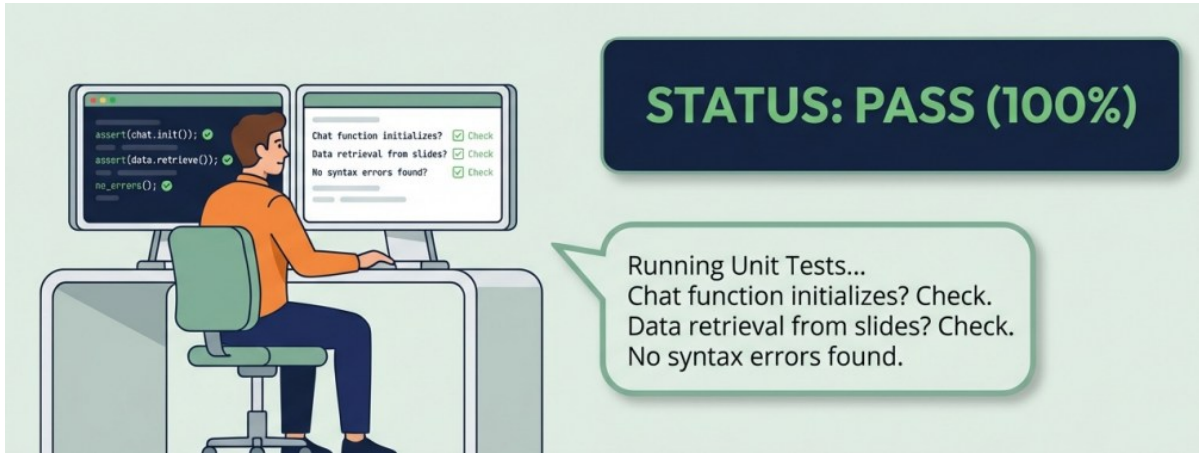




- The software functions correctly according to the software specifications
  - E.g. The AI Tutor successfully lets students chat and generates questions from the source text, exactly as the requirements document demanded."

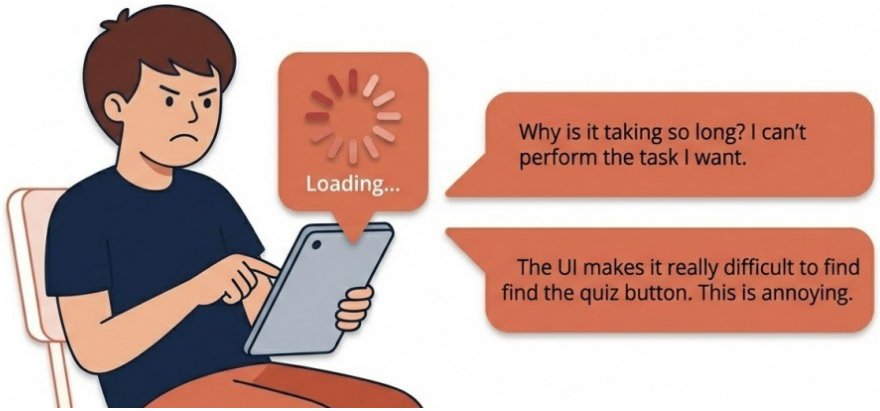






- When we move from the code editor to the real world …
  - "It involves meeting with real stakeholders (teachers) and users (students) to understand their pain points and discuss the prototype.“



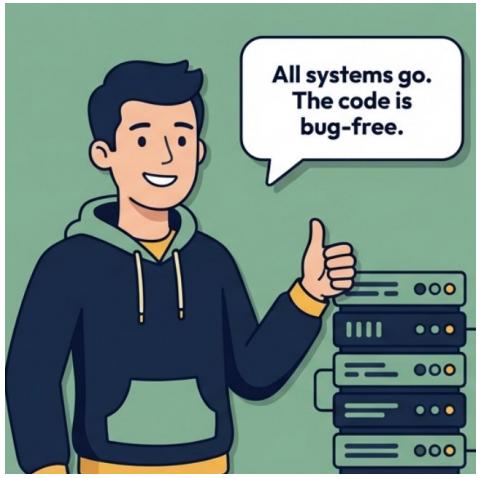


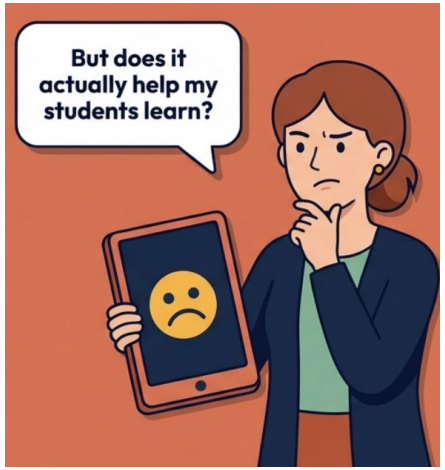




Verification vs. Validation

- **Verification** is a process of evaluating the software products of a software development lifecycle to check if we are on the right track to create the final product.
  - E.g, Code Review, Unit Testing, Integration, System Testing
  - Typically done by the developer to ensure that the software meets specified requirements and is correctly implemented.
  - Making sure we build the product right
- **Validation** is the process of evaluating the software product to check whether the software meets the user/business needs
  - E.g, Prototyping, Acceptance testing, Usability Testing, Testing in production, A/B Testing
  - Making sure we build the right product





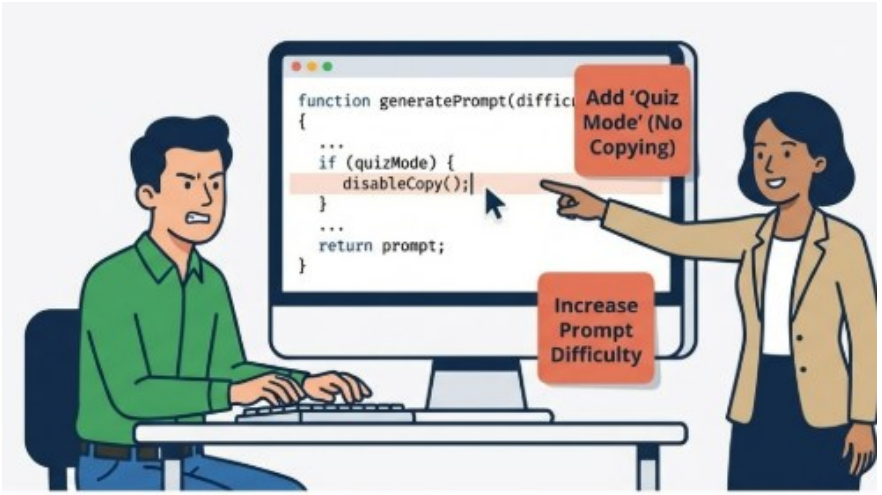




Feedback loop

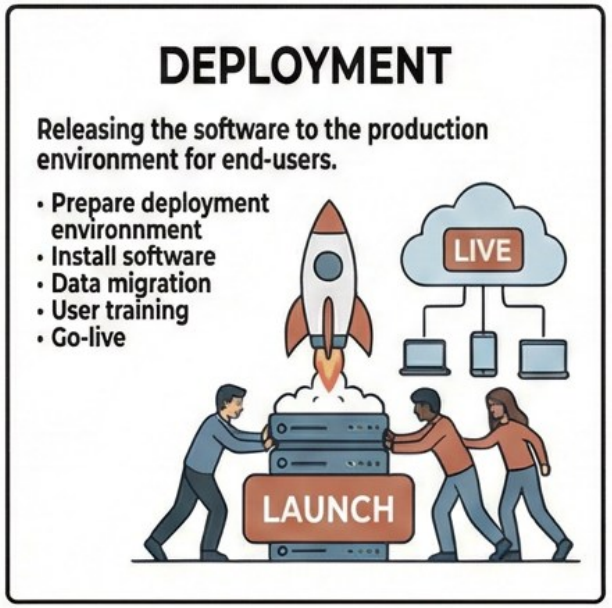
- Validation leads to new requirements for improving out software

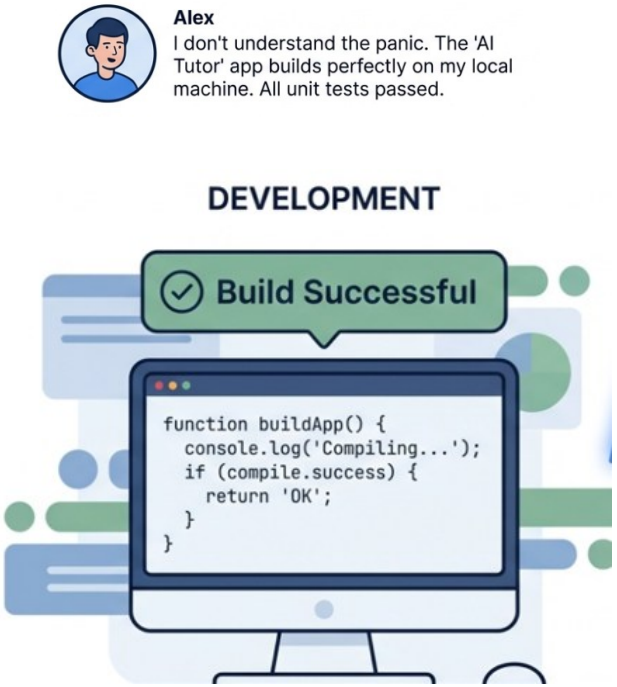




Software Deployment

- The software is deployed in the customer environment or released into the market/production.







Dev vs. Production environment

- Different OS versions, libraries
- Production data may contain edge cases or volumes not present in dev.
- Production environment may have stricter access controls or firewalls.
- Production servers may have different resource limits
- Production might rely on live external services that behave differently

Example: Convert pptx to PDF

```python
import comtypes.client
import os

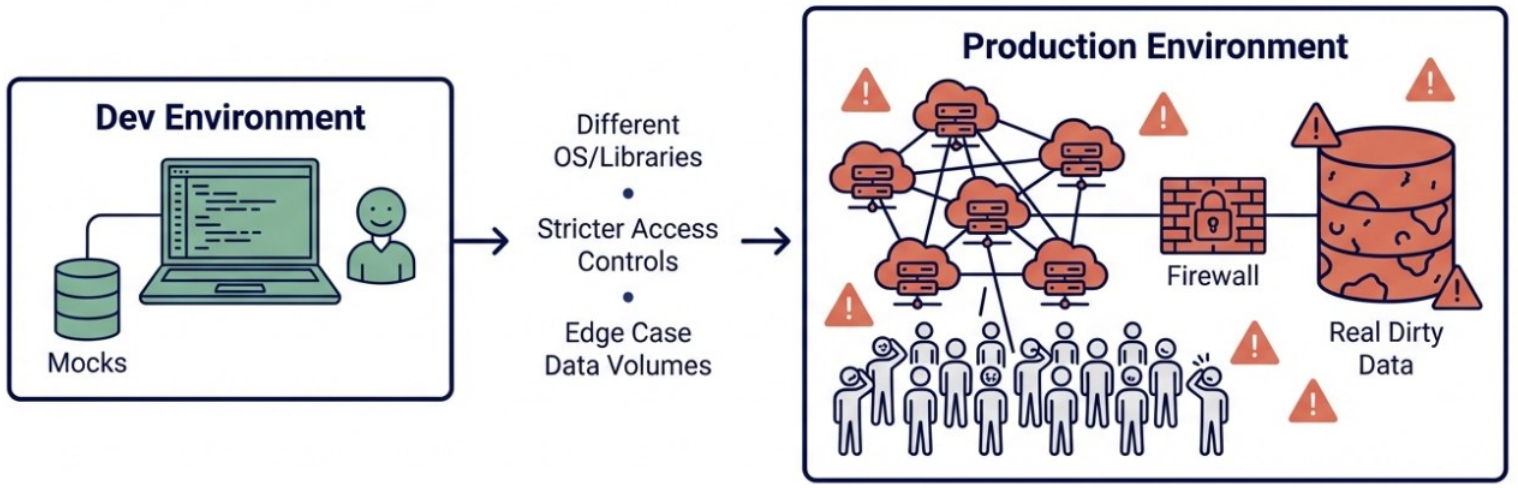
def pptx_to_pdf(input_pptx, output_pdf):
    # Initialize COM
    powerpoint = comtypes.client.CreateObject("PowerPoint.Application")
    powerpoint.Visible = 1

    # Open the presentation
    presentation = powerpoint.Presentations.Open(input_pptx)

    # Save as PDF (format type 32)
    presentation.SaveAs(output_pdf, FileFormat=32)

    # Close presentation and quit PowerPoint
    presentation.Close()
    powerpoint.Quit()

if __name__ == "__main__":
    input_pptx = os.path.abspath("example.pptx")
    output_pdf = os.path.abspath("example.pdf")
    pptx_to_pdf(input_pptx, output_pdf)
    print("Conversion completed.")
```

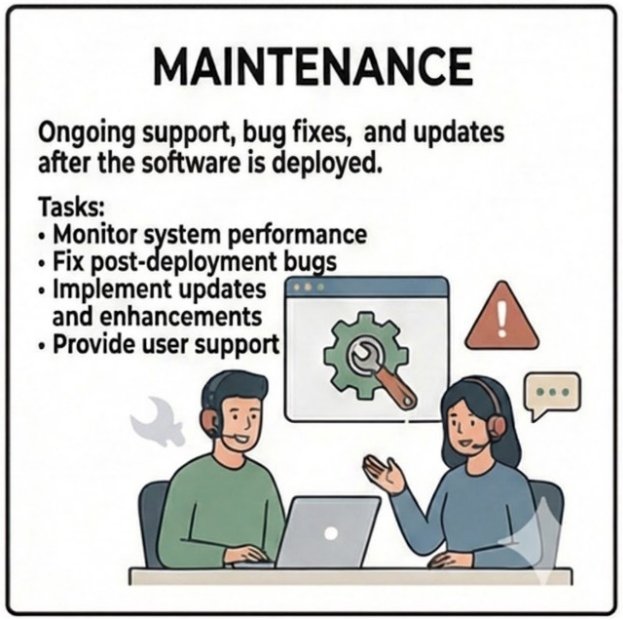


Video: Why Your Code Works Locally but Fails for Real Users?

Software Maintenance

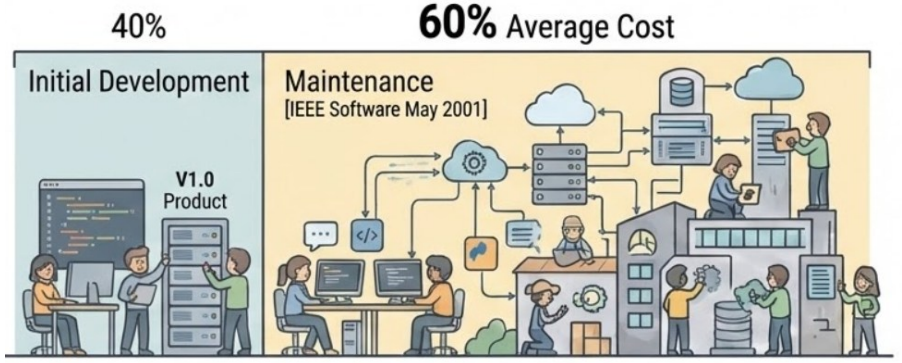
"Old hardware becomes obsolete; old software goes into production every night."
- Robert L. Glass

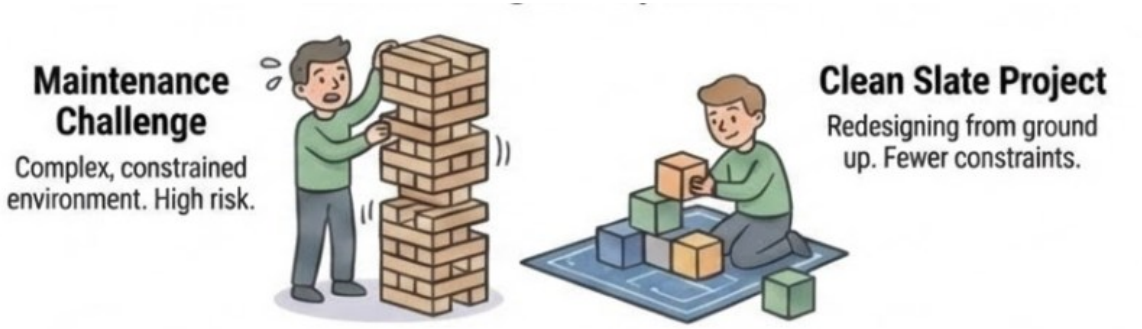
- Software maintenance in software engineering is the modification of a software product after deployment to production
- Tasks
  - Fix the bugs and issues in production environment, releasing patches, enhance the product and performance, release new versions with system enhancement, etc

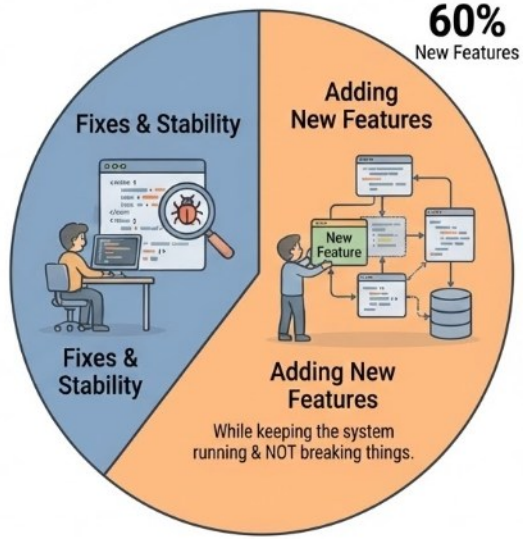


60/60 rule [IEEE Software May 2001]

- Maintenance can average 60% of software lifecycle cost
- About 60% of maintenance is to add new features, while keeping the system running and without breaking things.
- Maintenance is often more challenging than development in Clean Slate Projects (system designed from the ground up)!







ISO/IEC 14764:2006

- Provides guidelines for software maintenance
  - **Corrective:** Fixing defects found in the software.
  - **Adaptive:** Modifying software to remain compatible with a changing environment.
  - **Perfective:** Enhancing performance or maintainability.
  - **Preventive:** Addressing potential issues before they become problems.



Corrective Maintenance

Fixing defects found in the software.

E.g., Resolving a crash after a new feature release.



Adaptive Maintenance

Modifying software to remain compatible with a changing environment.

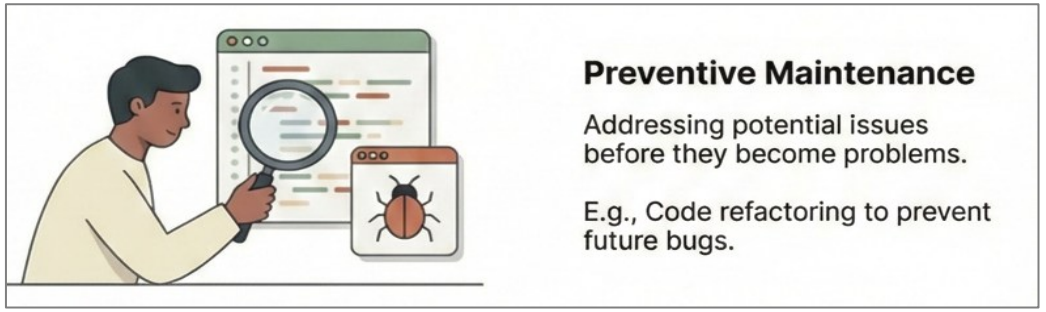
E.g., Updating for a new iOS version.



Perfective Maintenance

Enhancing performance or maintainability.

E.g., Improving load times.



Preventive Maintenance

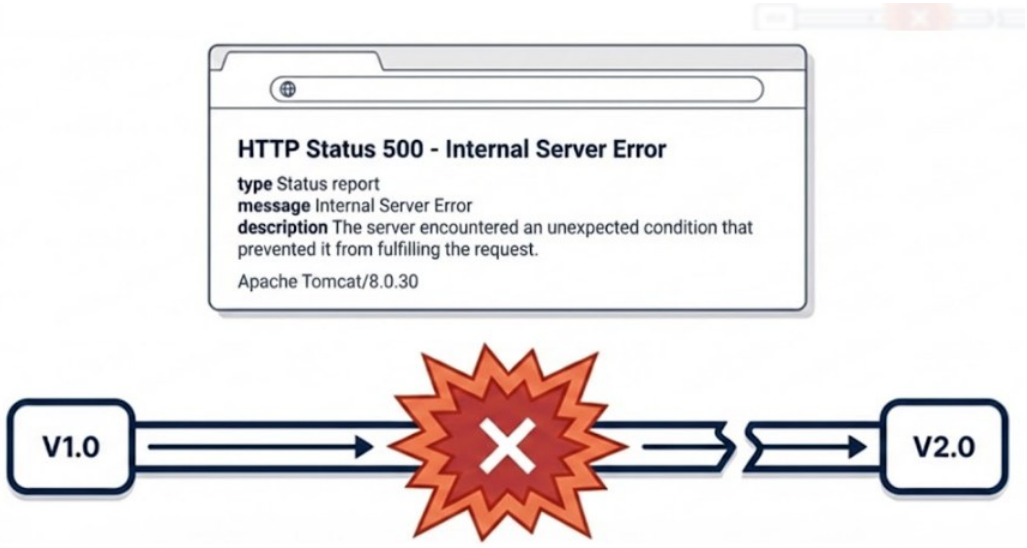
Addressing potential issues before they become problems.

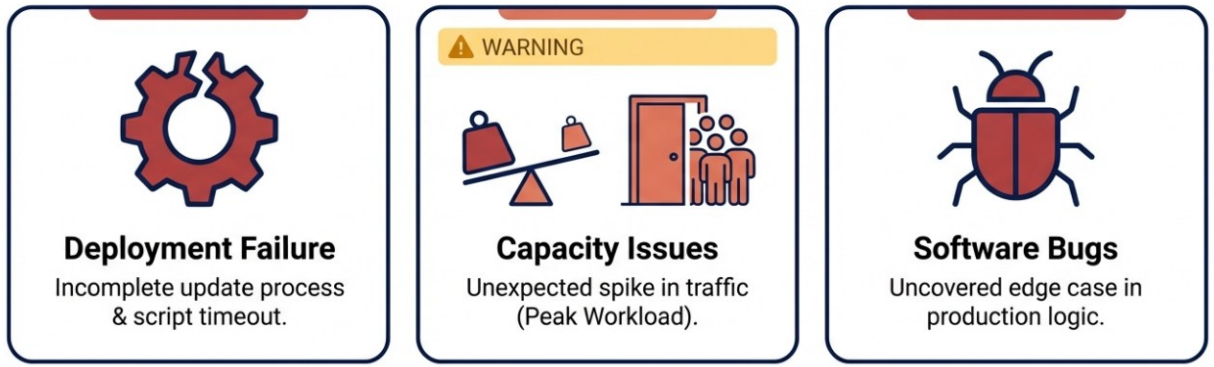
E.g., Code refactoring to prevent future bugs.

ISO/IEC 14764:2006
Software Engineering — Software Life Cycle Processes — Maintenance

Software Deployment Failure

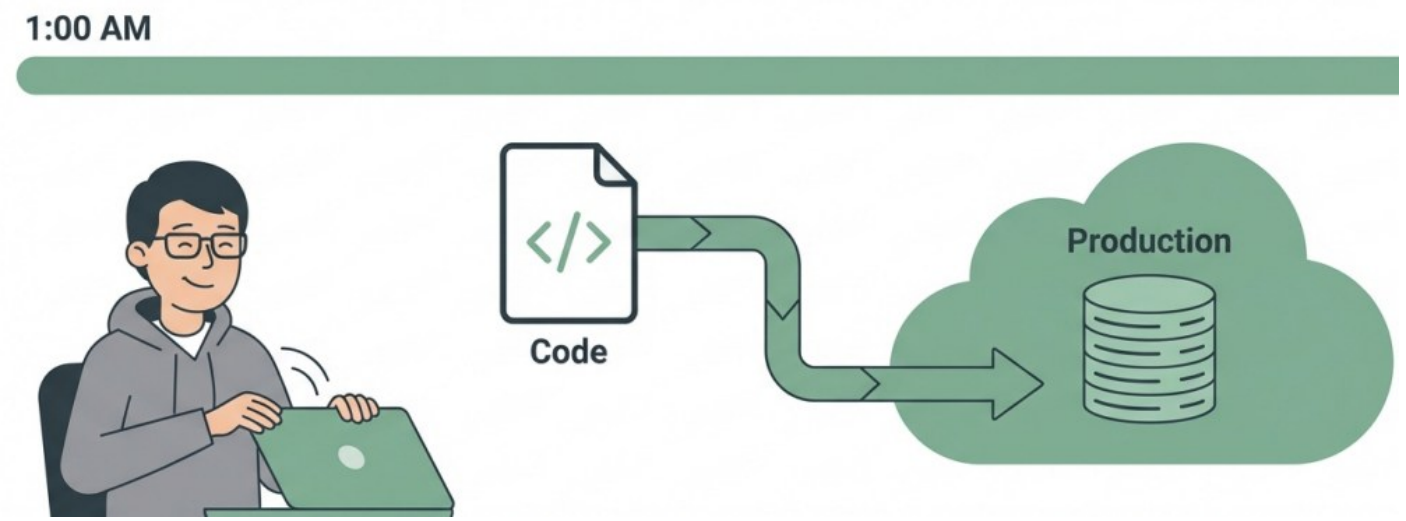
- The unsuccessful process of releasing and installing a software application or update into a target environment (e.g., production).
  - The software fails to start or stops responding soon after deployment.
  - Deployment logs or user interfaces show errors, exceptions, or failure codes.
  - Features do not work as intended or produce incorrect outputs.
  - The application runs significantly slower or consumes excessive resources.



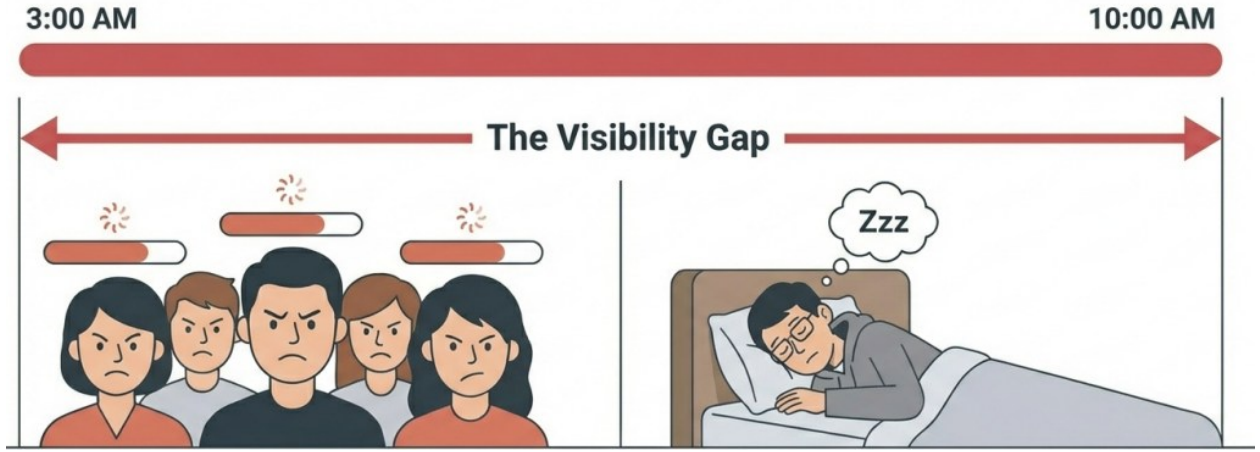


A Hypothetical Scenario

- Initial Deployment
  - At 1:00 AM, a new feature deployment is completed. All immediate health checks pass, and the system appears stable.



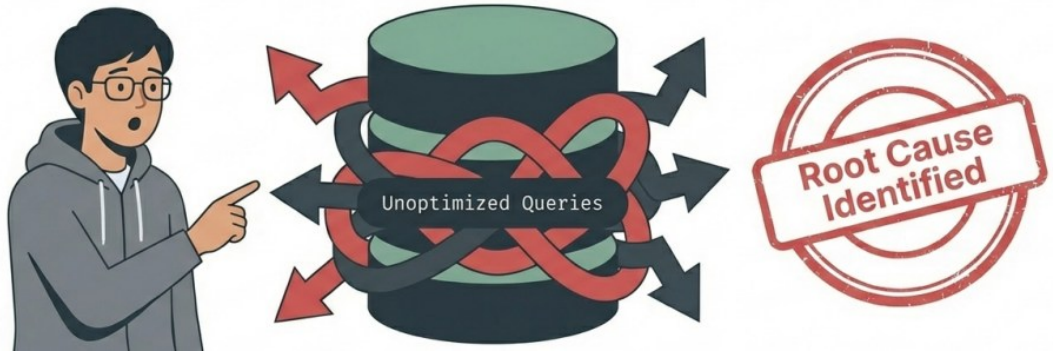
- Unnoticed Degradation
  - At 3:00 AM, two hours post-deployment, the new queries begin to cascade. Response times degrade significantly.
  - This is the start of the incident, yet there are no alarms.
- Discover the incident at 10:00 AM
  - For seven hours, the application is effectively unusable, yet operations teams are unaware.
  - The system is down, but the incident hasn't officially started for the team.





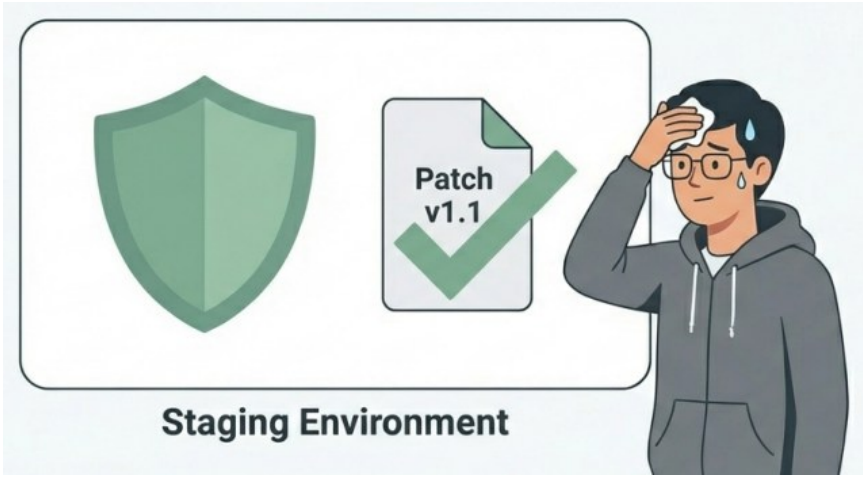
- Investigation of application and database logs reveals the symptoms: resource exhaustion on app servers and high latency.
  - The team correlates these metrics with the 1:00 AM deployment, narrowing the search field.
- Root cause:
  - The new feature introduced inefficient queries lacking proper indexing.
  - These queries are locking resources, causing the system-wide slowdown.



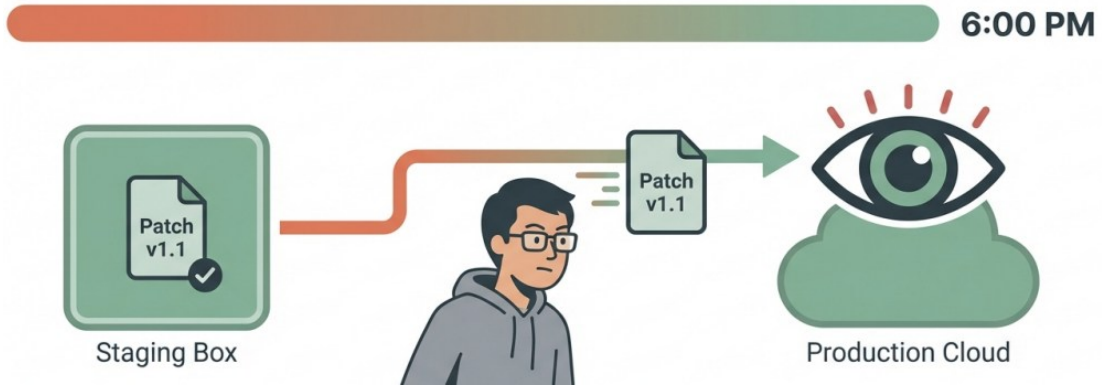


- Mitigation
  - The team begins rewriting the problematic queries and adding necessary indexes.
  - This phase is the 'work' of the incident. Speed depends on code complexity and engineering skill
- Patch v1.1 / Staging Environment
  - To prevent a regression or secondary failure, the patch is tested in a staging environment. Results confirm improved response times without errors.





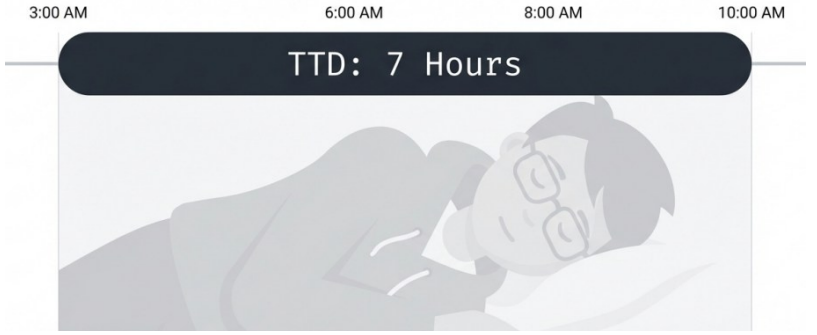
- The fix is deployed to production.
  - Patch v1.1 deployed to production
- The team watches real-time metrics to ensure the database load drops and user response times recover.
- Service restoration at 9 pm
  - Application response times stabilize to normal levels.
  - Customer complaints cease.
  - The incident is officially closed 18 hours after the performance degradation began.





Metrics

- **Time to Discover (TTD)**
  - The time elapsed between the start of the failure (3:00 AM) and the moment the team becomes aware of it (10:00 AM).
  - 3:00 AM to 10:00 AM = TTD: 7 Hours
- **Time to Mitigate (TTM)**
  - Measures the team's operational velocity
    - How fast they can diagnose, fix, and verify a solution once the problem is known.
  - The complexity of query optimization drove this 8-hour window.
- **Time to Recover/Repair (TTR)**
  - The total duration of impact on the customer.
  - While the team only 'worked' for 8 hours, the customer experienced 18 hours of pain.
  - 3:00 AM to 9:00 PM = Total Duration (TTR): 18 Hours







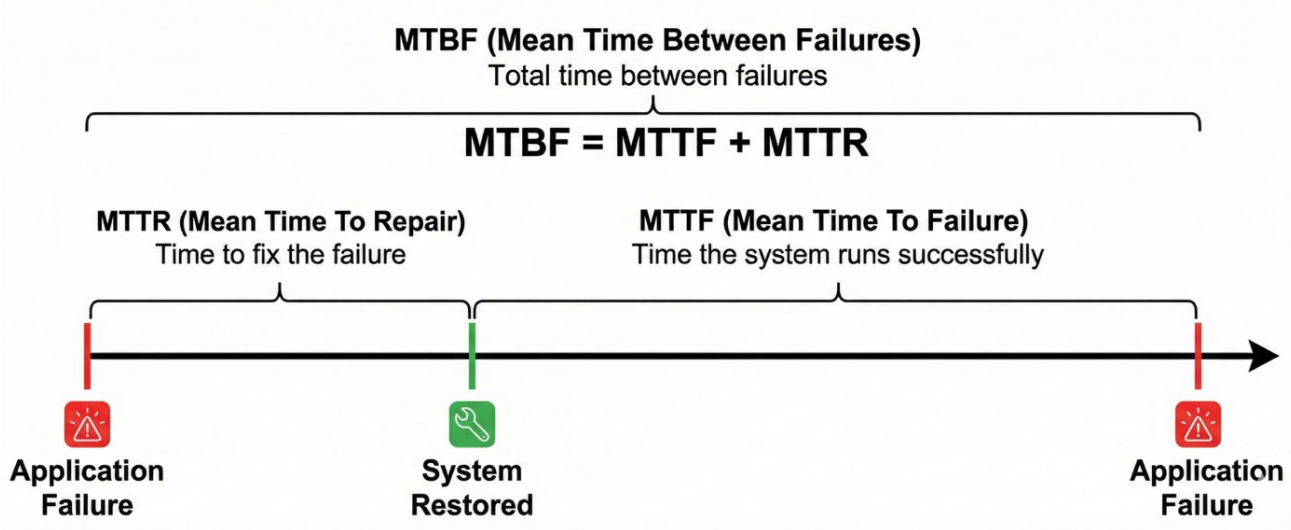
System Availability

$$\textbf{Availability} = \frac{\textit{Uptime}}{(\textit{Uptime} + \textit{Downtime})}$$



- The proportion of time that the system is functional and working.
- Measured as a percentage of uptime.
- Availability is often described in terms of "9s"
  - E.g., "four 9s" means 99.99% uptime.

% Uptime	Downtime per week	Downtime per month	Downtime per year
99%	1.68 hours	7.2 hours	3.65 days
99.9%	10 minutes	43.2 minutes	8.76 hours
99.95%	5 minutes	21.6 minutes	4.38 hours
99.99%	1 minute	4.32 minutes	52.56 minutes
99.999%	6 seconds	26 seconds	5.26 minutes



Availability = MTTF/(MTTF+MTTR)

What may affect the system’s availability?

Factor	Description	Impact on System Availability
Hardware Failure	Physical malfunction of servers, storage, or network devices	Causes downtime due to equipment outages
Network Instability	Unreliable or slow network connections	Leads to inaccessible or slow software
Software Bugs	Errors or defects in software code	Can cause crashes and unexpected shutdowns
Maintenance Downtime	Scheduled or unscheduled maintenance	Temporarily takes software offline
Security Attacks	Cyberattacks such as DDoS or malware	Disrupts service or forces shutdowns
Power Outages	Loss of electrical power without backup systems	Makes servers unavailable
Capacity Overload	Excessive user load beyond system capacity	Causes system crashes or degraded performance
Configuration Errors	Incorrect setup or changes in software or infrastructure	May prevent software from running correctly
Third-party Failures	Outages in external APIs or services	Causes interruptions if software depends on them
Poor Backup/Recovery	Ineffective data backup or slow recovery after failure	Prolongs downtime during incidents

Example: Tendering document for a virtual patient simulation platform

.3 Mandatory Services

6.3.1 Platform Architecture and Deployment

a) Scalability and Performance:
The AIVP platform should be in secure environments. Platform hosting must ensure that data is encrypted with high availability and redundancy. Platform hosting must accommodate fluctuations in user traffic and workload demands. Tenderers should show their platform have the capabilities of scalability, fault isolation and independent service update.

Estimated interaction volumes:
- Low Range: Less than 5,000 interactions per month
- Mid-Range: 5,000-10,000 interactions per month
- High Range: 10,000+ interactions per month

Text processing support: ≥250 traditional Chinese characters or 500 English characters per interaction.

First-response latency under 500ms; maintain operational performance under peak load.

b) Availability Requirements:

- 24 x 7 x 365 availability except for scheduled routine maintenance
- Support hours: 9:00 AM to 6:00 PM Hong Kong time, Monday to Friday except public holiday and adverse weather and extreme conditions.
- Minimum one-week advance notice required for scheduled downtime

c) Data Protection:

- Encrypt all data in use, in transit, and at rest (AES-256, TLS 1.2+).
- Platform must be protected against OWASP Top 10 web application security risks.
- First release and following annual Security Risk and Assessment with Penetration Testing mandatory.
- Implement rigorous backup and off-site recovery procedures; restore within 24 hours of failure.

d) Access Controls and Auditing:

- Enforce role-based access control (least-privilege).
- Log all access and configuration changes; retain logs for at least one year.
- Access logs provided to HKAM upon request

f) Data Protection:

- Rigorous backup and recovery procedures
- Platform failure recovery: restoration within 24 hours of failure occurrence
- Off-site backup must adhere to HKAM security requirements

g) Support Response and Resolution:

- Critical incidents: ≤ 4-hour response, ≤ 24-hour
- High: ≤ 8-hour response, ≤ 48-hour resolution
- Medium: ≤ 16-hour response, ≤ 96-hour resoluti

5

f) Multilingual and Multimodal Support
- Localize scenario text and AI prompts into Cantonese, Mandarin, and English.
- Inject TTS audio synchronized with avatar lip-sync if applied.

g) Performance and Scalability
- Library UI must render 45+ scenarios with < 500ms in first response times under 200 concurrent users.
- Backend must auto-scale to accommodate batch imports and heavy authoring workloads.
- Provide JMeter script and assist the testing during the UAT.

h) Support and Training
- Provide detailed user documentation, annotated examples of branching scenarios, and step-by-step video tutorials.
- Deliver two half-day live workshops for scenario authors and one train-the-trainer session.

7

6.3.2 Multimodal Patient Interaction

The AIVP Platform should support rich, interactive trainee-patient encounters by processing and responding to multiple input and output modalities (text, voice, and images/video) with low latency and high fidelity.

a) Languages & Speech Recognition
- Support voice input in Cantonese (HK/GD dialects), Mandarin, and English with ≤10% word-error-rate (WER) under quiet conditions.

b) Text & Multimedia Input
- Accept free-text queries with adaptive autocomplete and spell-check.
- Accept uploaded images/videos; auto-screen and inject context into responses.

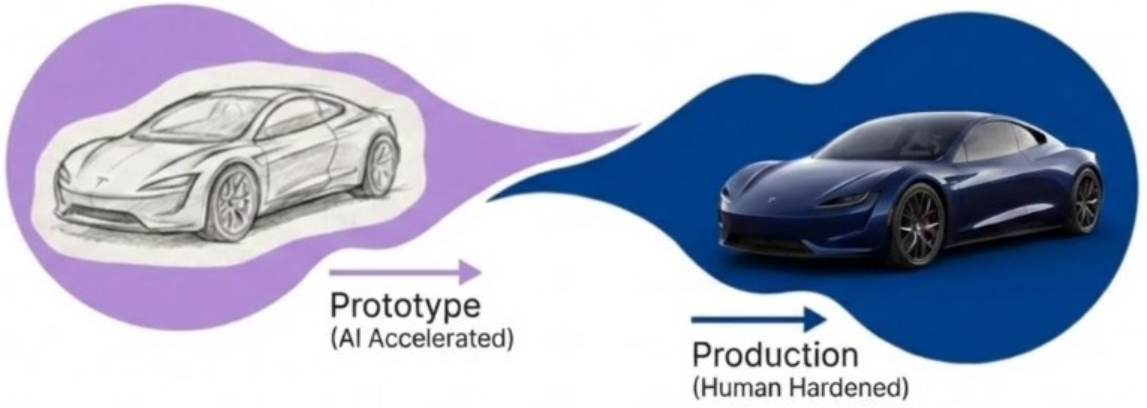
c) Dynamic Content Repository
- Provide centralized, versioned storage of scenario assets (2D/3D models, audio clips, images, videos) with tagged metadata for on-demand injection.

d) Real-Time Synchronization & Performance
- Maintain media-sync (avatar animation to audio) with end-to-end jitter ≤ 50ms.
- Automatically degrade gracefully to audio-only or text-only mode if network conditions fall below defined thresholds (configurable), with user notification.

e) Acceptance Criteria
- English speech recognition accuracy ≥ 90%
- Mandarin speech recognition accuracy ≥ 90%
- Cantonese speech recognition accuracy ≥ 80%
- Dynamic asset retrieval API < 100ms response.

Prototype vs. Production ready software

- Vibe coding can feel fast early (often the first ~70%), then progress collapses as complexity grows.
- However, the last mile is hardest: integration, edge cases, reliability, performance, security, deployment.
  - AI is not yet good at handling complex codebases and brownfield projects
- Production-ready software is difficult to complete without human expertise and engineering judgment.
  - Maintenance, security, scalability, and compliance, which requires careful human review and guidance



Software Deployment strategies

- "Big Bang" Deployment
- Rolling Upgrades
- Blue-Green Deployment
- Canary Deployment

"Big Bang" Deployment

- Roll out a new version in one go with service downtime.
- Roll back to the previous version if the deployment fails.
- Risky for public-facing or business-critical applications
  - Required the business to conduct extensive development and testing before release
  - Downtime without easy rollbacks.

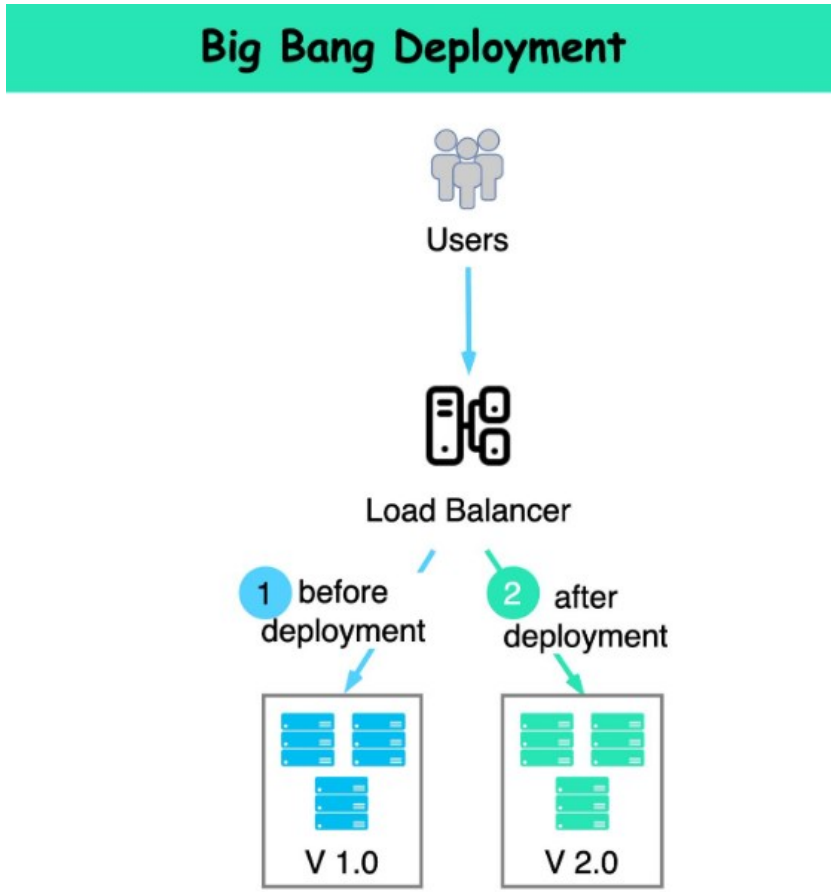
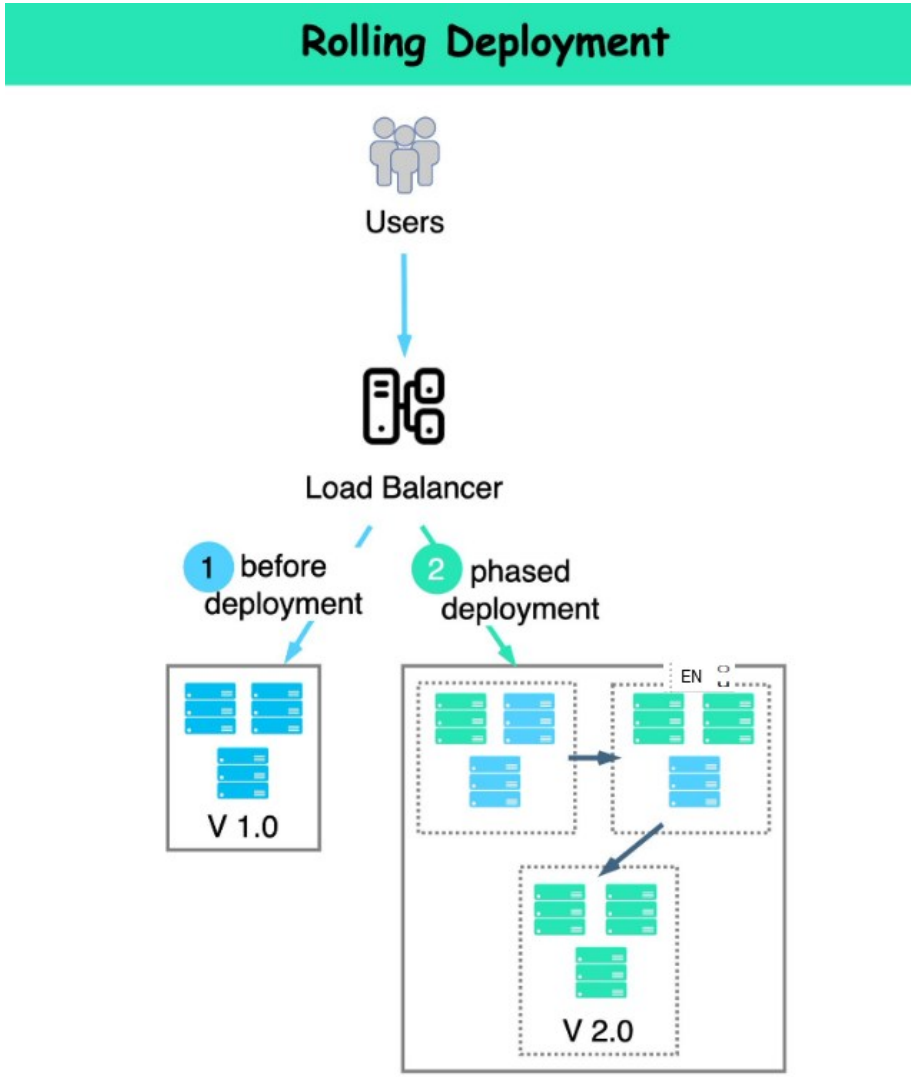


Image Credit: @alexxubyte on Twitter

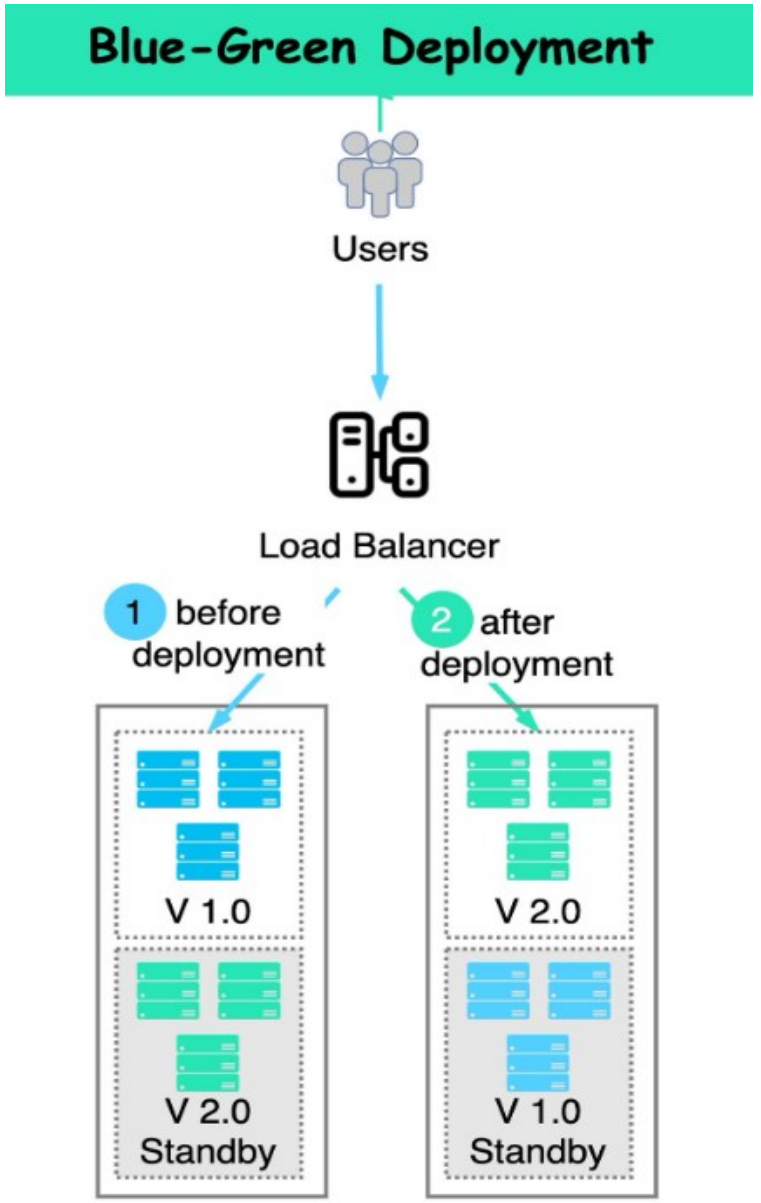
Rolling Upgrades

- An application's new version gradually replaces the old one.
- New and old versions will coexist for some time
- Allow roll back any new component incompatible with the old components.



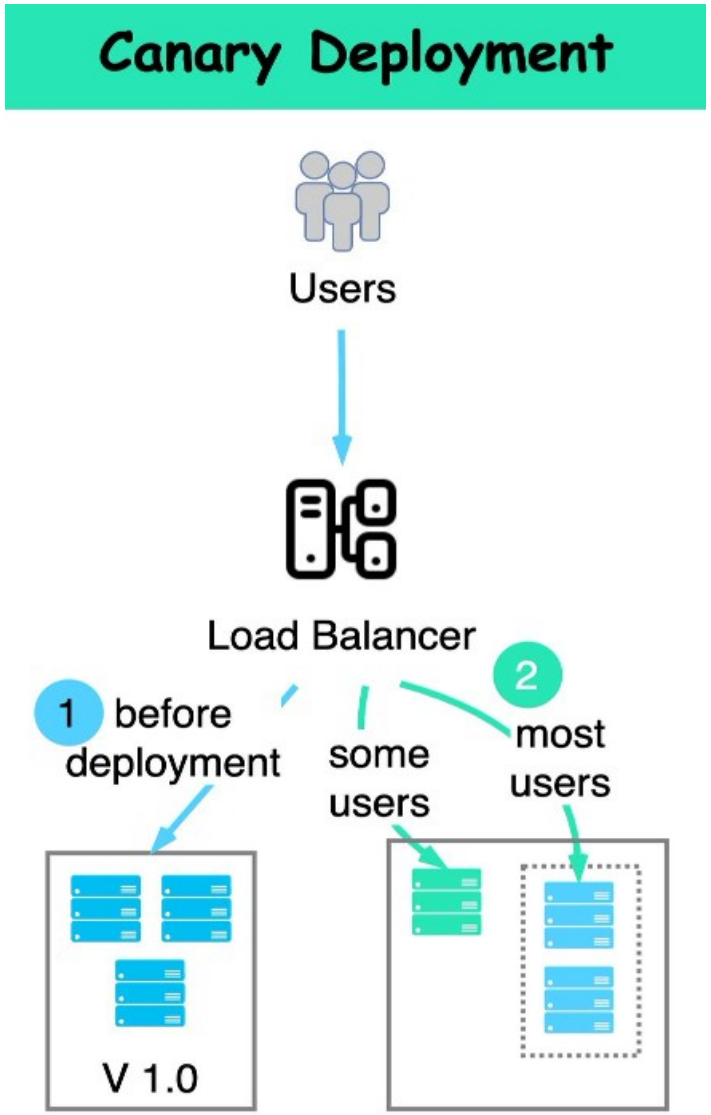
Blue-Green Deployment

- Maintain two identical production environments work in parallel.
  - One is the currently-running production environment receiving all user traffic (Blue).
  - The new version of the application is deployed in the green environment and tested for functionality and performance
- Once the testing results are successful, application traffic is routed from blue to green. Green then becomes the new production.
- If there is an issue after green becomes live, traffic can be routed back to blue.



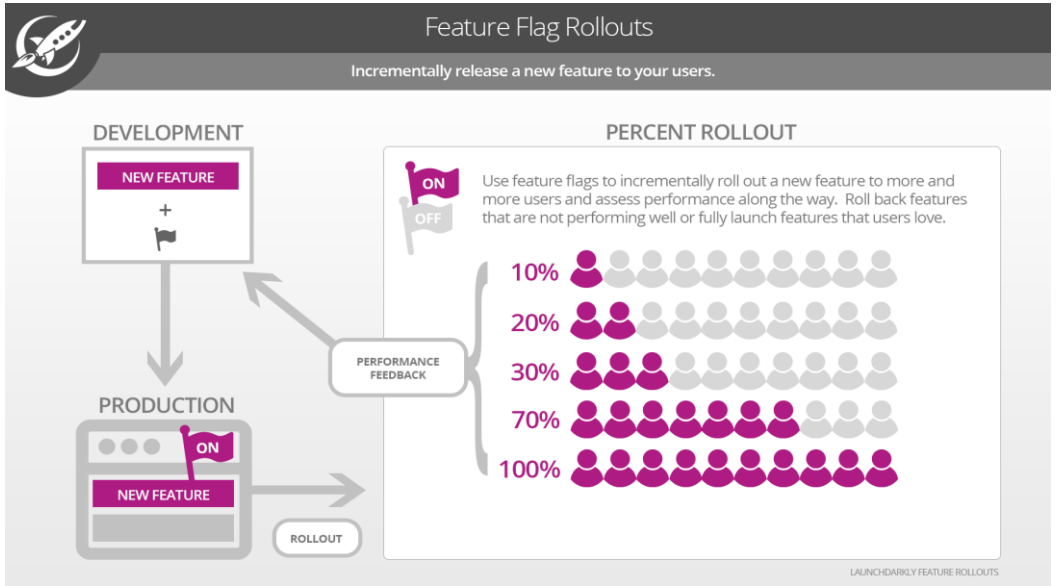
Canary Deployment

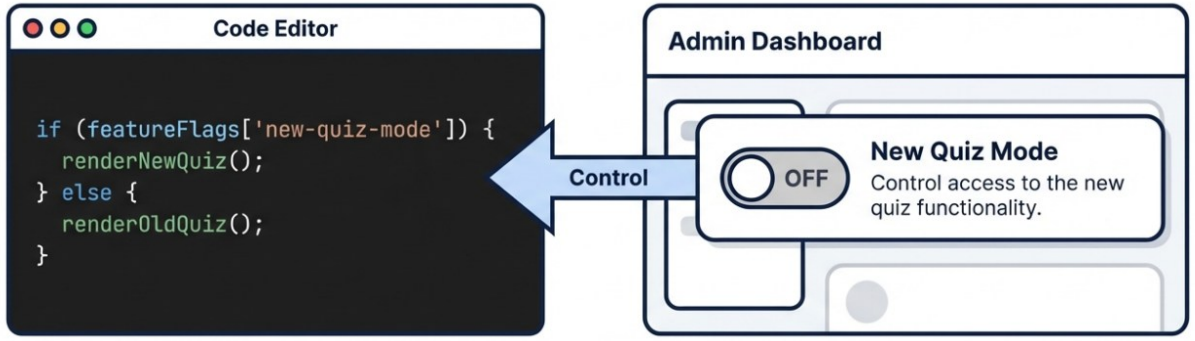
- Only a small portion of instances are upgraded with the new version
  - a portion of users are routed to canary instances.
- With no errors reported, the new version can gradually roll out to the rest of the infrastructure.
- Reduce the risk of introducing a new software version in production

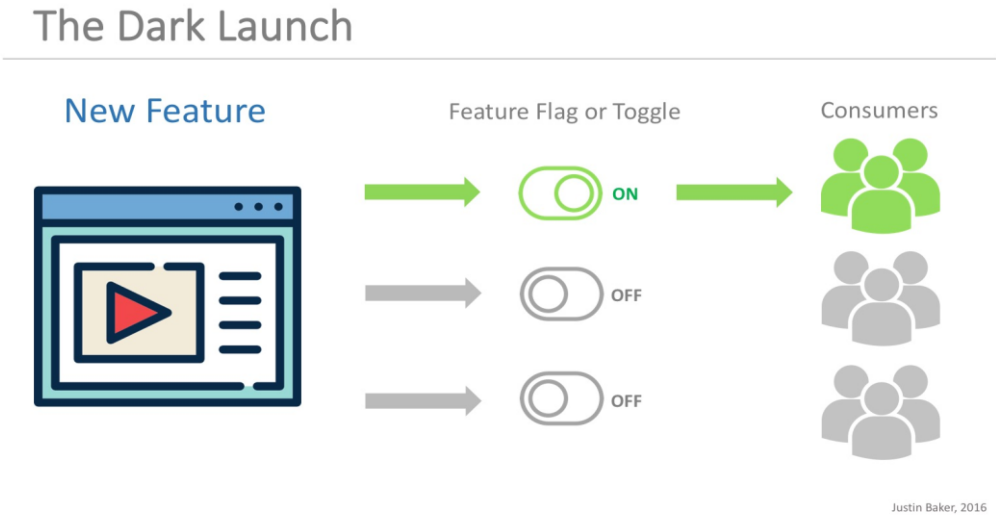


Feature Flag

- Used to control the visibility and activation of specific features in an application without deploying new code.

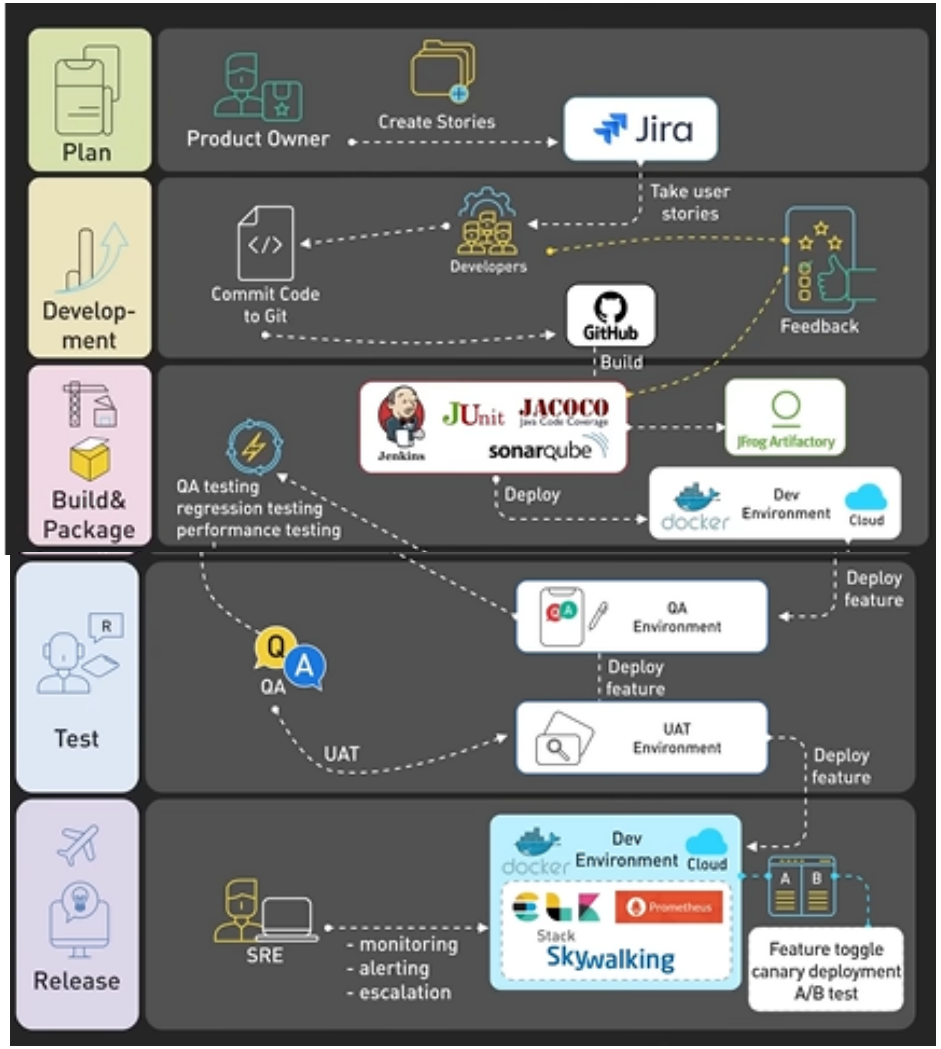






How Big Tech Ships Code to Production

- **Initial Development and Version Control:** During the coding phase, developers utilize Git to regularly commit code and track every change made to the project.
- **Continuous Integration/Continuous Deployment (CI/CD):** Once code is committed, a CI/CD server such as Circle CI takes over. This tool automates the building of the code, runs unit tests, and performs essential code quality checks.
- **Testing Environments:** Before going live, code is moved to separate QA environments where quality assurance teams perform regression and performance checks.
- **Production Rollout Strategies:** To ensure a smooth release into the live production environment, companies use specific deployment strategies and tools such as feature toggles, Canary deployments, and AB testing.
- **Monitoring and Reliability:** After the code is live, Site Reliability Engineers (SREs) use monitoring tools like Prometheus and Datadog to keep a close eye on system performance and resolve any emerging issues immediately



https://www.youtube.com/shorts/551lh10g_go